

Objetivos de la materia:

- ¿Cómo se administra la CPU y MEMORIA?
- ¿Cómo se gestiona la E/S?
- ¿cómo se modela para poder funcionar?

Información del documento:

Se encontrarán muchas definiciones usadas en la materia, también algunos comandos usados en prácticas/parciales.

POSIX : Es un estándar que define cómo debe comportarse un sistema operativo tipo Unix para que el software sea portátil y compatible entre distintas plataformas.

TEMA 1: Introducción 1

Perspectivas:

De arriba hacia abajo(USUARIO):

Windows no existe sin interfaz gráfica(ventanas) en cambio linux existe sin interfaz gráfica(console) y luego se le agrega la interfaz.

De abajo hacia arriba(SISTEMA):

En esta perspectiva se ve al SO como un administrador de recursos de hardware que provee servicios para poder usar esos recursos de hardware.

Maneja la memoria, los procesos compartiendo la CPU en el tiempo entre diferentes procesos y programas garantizando de una forma eficiente y segura.

Objetivos del SO:

Comodidad: Hacer más fácil el uso del hardware

Eficiencia: Hacer un uso más eficiente del manejo de los recursos del sistema

Evolución: Permitir nuevas funciones al sistema sin interferir con funciones anteriores.

Componentes de un SO:

1. Kernel
2. Shell

Tipos:

CUI: Interfaz de usuario basada en caracteres.

GUI: Interfaz gráfica de usuario. Navegadores web

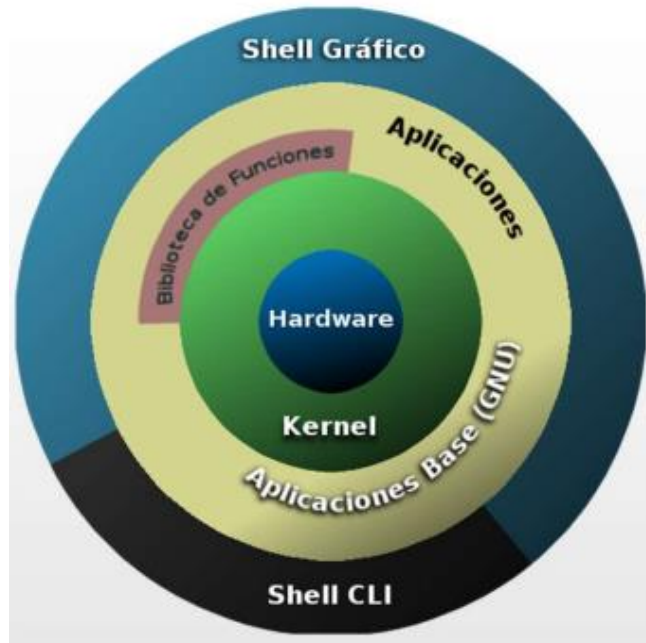
CLI: Interfaz de línea de comandos. PowerShell(win), cmd, bash(linux).

Herramientas:

Editores

Compiladores

Librerías



Esta estructura básica del GNU/Linux, está basada en una arquitectura en capas, donde cada nivel cumple una función específica y se apoya en el inferior. Sus elementos son:

Hardware: Ya viene de fábrica, el SO se adapta para poder usar ese hardware.

Ejemplo: CPU, memoria, discos, dispositivos, etc.

Kernel:

Es un Componente clave del SO, se encarga de la administración de recursos de hardware (CPU, memoria, dispositivos, discos).

Gestiona procesos, memoria, archivos y controladores.

El kernel es monolítico modular ya que se pueden agregar o quitar módulos(drivers) sin reiniciar y se encuentra siempre en la memoria principal.

Se comunica directamente con el hardware. Administra recursos y ofrece servicios básicos a las capas superiores.

Implementa servicios Esenciales(funciones):

Manejo de memoria(Asignar, proteger y liberar memoria para los procesos)

Manejo de la CPU(Decidir qué procesos se ejecuta)

Administración de procesos(creación, finalización, sincronización, comunicación)

Comunicación y concurrencia (permite que varios procesos se ejecuten en paralelo donde el kernel le da tiempo de CPU a cada proceso y esos procesos no trabajan aislados entonces el kernel provee mecanismos de comunicación para intercambiar información y coordinarse)

Gestión de E/S (coordinar acceso a dispositivos y periféricos)

Aplicaciones, son los programas que permiten al usuario interactuar con el sistema (como editores, gestores de archivos, navegadores, compiladores, etc.). Incluye aplicaciones gráficas (entornos de escritorio como GNOME o KDE).

Por ejemplo: El explorador de archivos, le pide al kernel abrir, mostrar el contenido de alguna partición o disco

Shell, es la interfaz entre el usuario y el kernel. Traduce los comandos que el usuario escribe y los envía al Kernel.

Por ejemplo: Explorer.exe o explorador de archivos es el Shell de win y la interfaz de comando BASH es el shell de linux.

Componentes fundamentales del gráfico:

kernel, Shell, Aplicaciones

Linux, es un kernel y tiene diferentes distribuciones(ubuntu, debian, etc) encargadas de agregarle información adicional a ese kernel.

Distribuciones, es una customización de GNU/LINUX formada por una versión de Kernel y determinados programas con sus configuraciones

Por ejemplo: UBUNTU, Es una distribución que depende de DEBIAN.

Servicios de un SO

Administración y planificación del procesador

Administración de memoria

Administración de almacenamiento

Administración de dispositivos

Detección de errores y respuestas

Interacción del usuario(shell)

Contabilidad

-

EJEMPLO de un proceso (Encender la PC, abrir CHROME y cerrar CHROME)

1) La CPU ejecuta el firmware BIOS/UEFI e inicializa el HW (CPU)

2) La BIOS carga el kernel del SO en memoria

3) El Kernel arranca los procesos base

- INIT o SYSTEMD en LINUX

- SYSTEM, SMSS.EXE, CSRSS.EXE, WININIT.EXE, EXPLORER.EXE, etc

4) Se mantiene el sistema vivo: Planificador, manejador de sesiones, servicios, etc.

5) Se hace clic en CHROME, el SHELL gráfico de win (explorer.exe que está en modo usuario) llama al kernel a través de una SYSCALL createProcess() o fork() exec() y esto genera una interrupción y la CPU cambia a modo kernel para que cree un nuevo proceso

6) Explorer.exe es el proceso padre del proceso CHROME

7) El kernel atiende la SYSCALL y prepara el contexto del proceso, crea la entrada del nuevo proceso en la tabla de procesos (PCB) y asigna esa entrada, carga el ejecutable CHROME.EXE en memoria, crea un espacio de direcciones de usuario (memoria virtual) y carga el programa de usuario ahí, configura el PSW con el bit de modo a usuario y el PC que apunta a la primera instrucción de CHROME. Prepara el contexto para el retorno y CHROME está listo para ejecutarse pero falta cambiar a modo usuario

8) El kernel usa la instrucción de retorno especial IRET para restaurar el PSW anterior o el PSW del proceso de usuario

9) El HW por medio del kernel ejecuta IRET y cambia automáticamente al modo usuario y el bit de PSW queda en modo usuario y salta a la primera instrucción de CHROME

10) Comienza la ejecución del proceso de usuario CHROME, y se ejecutan las instrucciones básicas

11) Chrome necesita un servicio de SO, por ejemplo: guardar en disco un archivo que se descargó entonces se ejecuta la instrucción SYSCALL write() o writeFile() para delegar la tarea al kernel, se genera una interrupción o trap y el HW cambia a modo kernel, guarda el contexto del proceso y ejecuta el manejador de la SYSCALL. Se ejecuta código privilegiado

12) SO termina, no se genera una interrupción solo se recupera el contexto del proceso, el HW por medio del kernel ejecuta la instrucción de retorno especial IRET y restaura el PSW del proceso CHROME con el bit de modo a usuario y la CPU vuelve a modo usuario y CHROME continúa ejecutándose

- 13) CHROME finaliza, el usuario cierra el programa, entonces se ejecuta la instrucción SYSCALL de salida EXIT() en linux o EXITPROCESS() en win, se genera una interrupción y la CPU cambia a modo kernel, se marca el proceso como finalizado, se libera todos los recursos que chrome utilizaba (PCB, páginas de memoria, entradas en la tabla de procesos, archivos abiertos, etc), se informa al proceso padre Explorer.exe que CHROME finalizó, el kernel elimina completamente el proceso de su tabla de procesos.
- 14) Kernel vuelve al planificador para elegir otro proceso listo y realizar el ciclo de preparar el contexto del proceso elegido, ejecutar IRET para volver a modo usuario
-

Cambio de modo, ocurre dentro del mismo proceso, cuando necesita ejecutar o volver de una función del SO, lo realiza el HW (CPU) al detectar una interrupción o trap o la instrucción IRET.

Ejemplo de Chrome:

Cuando Chrome está en modo usuario necesita escribir un archivo

Se ejecuta write().

Se genera una interrupción (trap)

La CPU cambia a modo kernel.

El SO escribe en disco.

Cuando termina, el kernel usa iret

La CPU vuelve a modo usuario y Chrome continúa.

Cambio de contexto, ocurre cuando el SO suspende la ejecución de un proceso y reanuda otro proceso guardando el contexto del proceso actual, es una tarea del planificador.

Ejemplo de Chrome:

Chrome está ejecutándose, pero vence su quantum de CPU.

El temporizador genera una interrupción.

La CPU pasa a modo kernel.

El kernel guarda el contexto de Chrome (PC, registros, PSW, etc.).

El planificador elige otro proceso (por ejemplo, Word).

El kernel carga el contexto de Word.

Ejecuta iret

La CPU pasa a modo usuario y ahora se ejecuta Word.

En cambio de modo:

Qué cambia: El nivel de privilegio del CPU (intercambio entre modo usuario y modo kernel).

Quién lo hace: El CPU a nivel de hardware.

Cuándo ocurre: Durante llamadas al sistema (syscalls), interrupciones, excepciones (traps) o al ejecutar la instrucción iret.

Ejemplo práctico: Chrome ejecuta la función write() para guardar un archivo y el sistema pasa temporalmente a modo kernel.

En cambio de contexto:

Qué cambia: El proceso activo en el sistema, lo que implica guardar y cargar el contexto de ejecución completo de un programa.

Quién lo hace: El Kernel a nivel de software.

Cuándo ocurre: Por decisiones del planificador, interrupciones del reloj de hardware o la finalización de un proceso.

Ejemplo práctico: El sistema operativo pausa la ejecución de Chrome para darle el turno y ejecutar Word.

TEMA 1: Introducción 2

En esta parte se habla de que el SO además de administrar los recursos de HW.

Debe hacer:

Gestionar el uso de la CPU

Detectar intentos de ejecución de inst. de E/S ilegales

Detectar accesos ilegales a memoria

Proteger el vector de interrupciones así como la RAI (atención de interrupciones)

Debe evitar:

Un proceso se apropie por completo de la CPU

Un proceso intente ejecutar inst. de E/S

Un proceso intenta acceder a una posición de memoria fuera de su espacio declarado

El SO debe tener apoyo de HW donde se provee 3 características:

Modos de ejecución: La CPU tiene 1 bit de modo en el registro PSW que representa el modo actual en que la CPU se está ejecutando y el sistema arranca en modo supervisor.

Modo kernel, toda instrucción privilegiada es posible ejecutarla. Por ejemplo: inst de E/S, IN, OUT, inst que cambien la palabra de estado del procesador.

Modo usuario, se ejecutan instrucciones básicas o "normales" como AND, MOV, JUMP. Si se ejecuta CHROME en este modo se garantiza que no haga un comando de E/S sino que utilice sólo los comandos que el SO brinda de E/S.

CAMBIO DE MODO, cuando el SO se tiene que ejecutar se pasa al modo KERNEL y cuando los procesos se tienen que ejecutar se pasa al modo USUARIO. Para realizar este cambio se utiliza las interrupciones y cambia el estado del bit de modo.

Interrupción de clock: Es una interrupción constante que permite proteger la CPU para evitar que un proceso se apropie de la CPU por completo.

Se genera una interrupción:

La CPU pasa a modo kernel.

Se llama al vector de interrupciones correspondiente a la de clock.

Se ejecuta la rutina para gestionar esa interrupción: permite proteger las instrucciones de E/S, permite proteger que un proceso no se apropie de la cpu, proteger el espacio de direcciones.

Protección de la memoria: Cada proceso que se ejecuta tiene un límite de direcciones que puede utilizar. Por ejemplo, el SO asigna al proceso CHROME un espacio de memoria de A(reg. base) hasta C(reg. límite) y cuando el SO le da la CPU a CHROME la CPU está en modo usuario y el SO no está para controlar por eso se hace uso del apoyo del HW para controlar esto. Lo controla con el registro base y el registro límite

TIPO DE KERNEL

Monolítico, toda funcionalidad que debería o debe implementar el SO se ejecuta en modo kernel. La ventaja está en que implica menos tiempo en la resolución de tarea, ya que se cambia de modo rápido (PERFORMANCE DE EJECUCIÓN)

Microkernel, No todas las funciones del SO se ejecutan en modo kernel, crear procesos, planificar su ejecución donde se elige qué proceso va ejecutarse están a cargo de servidores de usuario comunicándose con el microkernel. La ventaja está en tener más seguridad

Iniciativa GNU se refiere a 4 libertades en el uso del software (LICENCIA)

- 1) Libertad de usar el programa con cualquier propósito
- 2) Libertad de estudiar su funcionamiento
- 3) Libertad para distribuir sus copias

4) Libertad para mejorar los programas

Características generales del SO GNU/LINUX de cualquier UNIX

Multiusuario

Multitarea y multiprocesador

Es altamente portable: Se puede ejecutar en una gran variedad de arquitecturas de hardware y dispositivos.

Estructura básica del Kernel

Ejecuta programas, gestiona dispositivos de hardware.

Sus funciones más importantes son la administración de memoria, CPU y la E/S.

Es un núcleo monolítico híbrido, los drivers y código del kernel se ejecutan en modo privilegiado.

El microprocesador cuando está ejecutando algo está en modo usuario o modo kernel(modos supervisor). Si el procesador está en modo usuario, está ejecutando una instrucción que corresponde a un programa de usuario.

Ejemplo: Asignar una variable en memoria. Cuando el proceso de usuario que se está ejecutando requiere acceder a una estructura de datos del kernel o acceder al hardware por una instrucción privilegiada(ej: para ver la hora) entonces el SO produce una interrupción y se activa el modo kernel delegando la operación de Lectura/Escritura al driver y el driver se ejecuta en modo supervisor para realizar esa Lectura/Escritura. Entonces si se tiene que acceder a un dispositivo se produce una interrupción, se ejecuta el SO y el SO delega la tarea requerida al driver asociado al hardware(ej: ver la hora del sistema y devolverle la hora al proceso). Hay un cambio de contexto.

Núcleo- versionado del SO

Tiene una nomenclatura de versionado de 4 dígitos A.B.C (.D)

A: Denota versión (1.0)... versión (2.0) ...

B: Denota mayor revisión, num pares desarrollo y num impares producción.
versión desarrollo(2.5) ... versión producción (2.6) ...

C: Denota una menor revisión. cambia cuando hay nuevos drivers o características

D: Cuando se corrigen algunas fallas

Sistema de archivos del SO

Organiza la forma en que se almacenan los archivos en dispositivos de almacenamiento.

El adoptado por GNU/LINUX es el sistema(Filesystem) EXTENDED que podría ser reemplazado por BTRFS de Oracle. Windows utiliza el filesystem NTFS.

IDENTIFICACIÓN DEL ARCHIVO

Se identifican por su NOMBRE, RUTA, TIPO determinado por su metadatos, PERMISOS Y PROPIETARIO, IDENTIFICADOR ÚNICO.

Cada archivo en Linux tiene un **nombre único dentro de su directorio** y se identifica mediante su **ruta absoluta o relativa** dentro de la jerarquía del sistema. /home/joa/documentos/informe.txt.

Tienen un número llamado **inode**, contiene toda la información del archivo excepto su nombre.

Por su tipo definido en su metadata

Filesystem, es la **estructura lógica y jerárquica** que utiliza GNU/Linux para **organizar, almacenar y acceder** a la información en el disco.

FHS(Filesystem Hierarchy Standard) determina cual es el propósito del árbol directorio del GNU/LINUX UNIX. Hay directorios que tienen usos específicos:

/tope: tope de la estructura de directorios. En win es C:\

/var: Información que varía en el tiempo. Ejemplo: Log de un programa

/home: Se almacenan los archivos de usuarios

/etc: Archivos de configuración

/etc/inittab: archivo que contiene información para el proceso INIT de que debe hacer y en qué orden. se define aquí los runlevels en system V

/etc/passwd: Contiene la **información básica de cada usuario** registrado en el sistema: nombre de usuario, x: aviso que la clave esta en etc/shadow, UID(id unico), GID(id de grupo), información detallada del usuario, directorio personal, shell.

Ejemplo: **carlos:x:1001:1001:Carlos Perez:/home/carlos:/bin/bash**

/etc/shadow: Contiene la **información encriptada de las contraseñas** de los usuarios y parámetros de seguridad. Solo puede ser leído por el **superusuario (root)**.

/etc/group: Contiene la información de los **grupos de usuarios**, como el nombre del grupo, su identificador (GID), x: aviso que la clave está en etc/shadow, y los usuarios que pertenecen a él.

Ejemplo: **nombre_grupo:contraseña:GID:lista_usuarios**

/bin: Archivos binarios y ejecutables. Guarda cualquier comando de usuario. Ejemplo: Cuando se ejecuta un comando en el intérprete de comandos. El SO busca en el /bin

/dev: Enlace a dispositivos

/dev/tty Dispositivo de caracteres

/dev/sda Dispositivos de bloques

/usr: Aplicaciones de usuarios

Notas:

1. Cuando se crea un usuario en el sistema, la información básica como el nombre de usuario, nombre personal, identificador único del usuario (UID), identificador del grupo primario (GID), directorio personal y shell se guarda en el archivo **/etc/passwd**.
2. La información de **todos los grupos del sistema**, junto con los usuarios que pertenecen a cada grupo como **miembros secundarios**, se almacena en el archivo **/etc/group**.

PASSWD: usuario:x:UID:GID:comentario:/home/usuario:/bin/bash

Otros Directorios:

/tmp lugar donde el SO y programa guardan archivos temporales

/boot lugar donde están los archivos necesarios para iniciar Linux antes de que el sistema esté cargado. Aquí están los archivos que el cargador de arranque (bootloader) necesita para iniciar el SO en este caso el GRUB.

/boot/vmlinuz-<versión-del-kernel> Ubicación del kernel

/lib se guardan bibliotecas compartidas y módulos del kernel

/lib/systemd/systemd ubicación del primer proceso systemd

/media lugar donde se conecta con dispositivos externos para acceder a sus archivos

/mnt se almacenan montajes temporales/manuales

/opt lugar donde están programas de terceros o opcionales que no forman parte del SO

/root es la carpeta personal del administrador del sistema o superusuario

/sbin se guardan comandos esenciales del sistema principalmente comandos que utiliza el administrador(reboot, shutdown). Aquí se encuentra el proceso INIT

/sbin/init Ubicación del proceso INIT

GRUB es un tipo de bootloader, es un programa que se ejecuta inmediatamente después de ejecutar el BIOS/UEFI y su función es cargar el kernel y la imagen initramfs. Luego el kernel busca los procesos INIT (sbin/init) o systemd (lib/systemd) para inicializar el sistema.

GRUB facilita la lectura de la filesystems (FAT, ext4, NTFS) para buscar el kernel

COMANDOS:

> redirige la salida de un proceso a un archivo o de un archivo a un archivo

ls > archivo.txt el contenido del directorio se almacena en archivo (creando el archivo si no existe o sobrescribiendo el contenido anterior)

pipe (p1|p2) comunica procesos. La salida de p1 es la entrada de p2

Cat sirve para mostrar contenido, concatenar, crear archivos de texto

cat archivo.txt Muestra el contenido del archivo

cat archivo1.txt archivo2.txt muestra el contenido de ambos archivos

cat archivo1.txt > nuevo.txt El contenido del archivo1 sobrescribe el contenido archivo nuevo

cat a.txt b.txt > c.txt concatena el archivo a y b y sobrescribe en c

cat archivo1.txt >> archivo2.txt El contenido del archivo1 se añade al final del archivo2

cat > notas.txt crea un archivo

touch notas.txt crea un archivo vacío si no existe, en caso de existir cambia la fecha de modificación del archivo.

less nombre_del_archivo ⇒ Muestra el contenido del archivo por página pero te permite volver hacia atrás y ver todas las páginas

more nombre_del_archivo ⇒ Muestra el contenido del archivo de una página a la vez

ls muestra lista de archivos y directorios del directorio actual

ls -l f2 Muestra la lista de archivos y directorios del directorio actual f2 detallando todos los permisos, propietario, tamaño, etc.

ls -ld f2 Muestra solo los detalles de permisos, propietario, tamaño del directorio f2 sin su contenido

ls > f1 guarda la lista de archivos y directorios del directorio actual en el archivo f1, sobrescribe lo anterior

ls >> f1 guarda la lista de archivos y directorios del directorio actual en el archivo f1, agrega al final de lo anterior

Grep busca patrones de texto dentro de archivos o en salida de otros comandos y las muestra en pantalla

grep -q "palabra" texto.txt busca la palabra en el archivo y No muestra las líneas que coinciden pero sí informa con un código de salida, usada para condicionales como IF

grep "palabra_a_buscar" carpeta/* busca la palabra en todos los archivos que tiene la carpeta y muestra las líneas que coinciden, ignora los subdirectorios

grep -r "palabra_a_buscar" carpeta/ busca la palabra en todos los archivos y subdirectorios que tiene la carpeta y muestra las líneas que coinciden,

grep "texto" archivo.txt busca "texto" tanto como palabra completa o palabra que contenga "texto " en el archivo y devuelve todas las líneas que coinciden. Eje: texto y pretexto, muestra las dos líneas)

grep -w "texto" archivo.txt busca solo la palabra exacta "texto" en el archivo y devuelve todas las líneas que coinciden.

grep -i "texto" archivo.txt busca la palabra en el archivo y muestra las líneas que coinciden sin distinguir mayus/minus

grep -n "texto" archivo.txt muestra el numero de linea donde aparece la palabra texto

grep -c "texto" archivo.txt muestra la cantidad de veces que se encontró la palabra texto

who | grep -w "ana" busca línea por línea la palabra exacta "ana" delimitada por limitadores como espacios, tabulaciones, inicio o fin de línea. Por último la imprime por pantalla

who | grep -q ana funciona como -w pero no imprime por pantalla, utilizada en IF cuando solo importa el código de retorno.

cp archivo1.txt copia.txt copia el contenido de archivo1 en copia en el mismo directorio

cp archivo.txt /home/usuario/ copia el archivo del directorio actual al directorio /home/usuario

cp /home/usuario/dir1/f4 /home/usuario/dir1/dir11/ copia el archivo f4 del directorio dir/ en el directorio dir11/

cp -r /var/log/* /tmp/logs/ copia el contenido del directorio /var/log/ en el directorio /tmp/logs/

mv /home/asdl/informe.txt /tmp/ mueve el archivo informe.txt de /home a /tmp/

mv "\$i" "\$HOME/bin/" en SCRIPT BASH, mueve el archivo de la ruta \$i hacia el subdirectorio /bin/

mv /home/usuario/dir1/f1 /home/usuario/dir1/f2 renombra el archivo f1 por f2

chown cambia propietario y/o grupo de un archivo o varios o directorios

chown [nuevo_dueño] archivo nuevo propietario del archivo

chown usuario:grupo archivo nuevo propietario y grupo del archivo

chgrp cambia el grupo propietario del archivo o directorio

chmod sirve para cambiar permisos de lectura, escritura, ejecución de archivos o directorios (linux tiene 3 tipos de permisos y 3 tipos de usuarios). Tener en cuenta que cuando se crea un archivo con nano, touch, vi, redirección tienen por defecto permisos 644(rw-r-r).

FORMA OCTAL:

(lectura=4 escritura=2 ejecución=1)

u:usuario; g:grupo; o:otros; **chmod ugo <archivo>**

chmod 755 mostrar.sh el dueño del archivo tiene todos los permisos y el resto pueden leer y ejecutar (rwx-rx-rx)

cd cambia de directorio

mkdir nom_carp crea un nuevo directorio en el directorio actual

mkdir /tmp/logs crea el directorio logs en el directorio tmp

mkdir -p "\$HOME/bin" en SCRIPT BASH crea el subdirectorio /bin/ en el HOME del usuario actual, -p permite crear en caso de no existir o evitar error si existe.

rmdir /tmp/logs elimina el directorio vacío

rm -r /tmp/logs elimina el directorio con contenido

rm -rf /tmp/* elimina solo el contenido del directorio

ln archivo.txt enlace.txt crea un enlace entre archivos

tail archivo.log muestra las últimas 10 líneas del archivo

head /etc/passwd muestra las primeras 10 líneas del archivo

locate notas.txt busca archivos por nombre en todo el sistema

find busca archivos en un árbol de directorios

find /home -name "notas.txt" busca el archivo notas.txt por nombre desde el directorio actual hasta sus subdirectorios y devuelve la ruta donde está ubicado o espacio vacío en caso de no encontrarlo

find . -name passwd busca el archivo passwd desde el directorio actual hasta sus subdirectorios y devuelve la ruta donde está ubicado o espacio vacío en caso de no encontrarlo

find / -name passwd busca el archivo passwd en todo el sistema de archivos y devuelve la ruta donde está ubicado o espacio vacío en caso de no encontrarlo

find . -name "*.txt" busca a partir del directorio actual todos archivos que sean .txt y devuelve las rutas donde están ubicados los archivos .txt o espacio vacío en caso de no encontrarlo

find . -name "*.txt" -o -name "*.pdf" busca a partir del directorio actual todos archivos que sean .txt y .pdf y devuelve la ruta donde está ubicado o espacio vacío en caso de no encontrarlo

find ~ -name "*.conf" busca en el directorio HOME del usuario que ejecuta el script todos los archivos ".conf" y devuelve la ruta donde está ubicado o espacio vacío en caso de no encontrarlo

pwd muestra la ruta del directorio actual del usuario actual

echo muestra texto o el valor de una variable

echo \$HOME muestra la ruta del directorio personal del usuario actual

man ls muestra el manual de ayuda del comando man

uname muestra información del SO y del kernel

dmesg muestra mensajes del kernel

lspci lista dispositivos del sistema (tarjeta gráfica, de red, de sonido)

at 10:30 programa la ejecución de un comando a las 10:30

netstat muestra información de la red

ps muestra una lista de procesos en ejecución del usuario actual

ps -ef muestra todos los procesos del sistema

ps muestra la lista de procesos que están corriendo en el sistema en forma de árbol para ver las relaciones padre-hijo

ps>f0 la lista de procesos se guarda en el archivo f0, sobrescribe lo anterior

ps>>f0 la lista de procesos se guarda en el archivo f0, agrega al final de lo anterior

halt detiene todos los procesos y apaga el sistema

kill pid para terminar un proceso enviando una señal

kill -9 pid termina el proceso de forma inmediata sin guardar datos, recursos, etc.

kill \$(pidof init) o kill\$(pidof systemd) busca el proceso init o systemd y lo termina

pkill firefox para terminar procesos por su nombre

killall chrome finaliza todos los procesos con el nombre chrome

top monitorea los procesos en tiempo real con su uso de CPU

htop además de monitorear los procesos, permite navegar, filtrar y matar procesos con teclas

atop es el top pero se le agregan opciones E/S de disco, red, uso de memoria.

pgrep Busca procesos por nombre parcial o completo y muestra los PID encontrados. A diferencia de pidof que busca el nombre exacto del proceso y devuelve un solo pid.

nice -n 10 comando ejecuta un proceso con menor prioridad

renice -n 5 -p 1234 cambia la prioridad de un proceso ya en ejecución

systemctl es la herramienta principal de administración del sistema de inicio systemd. Permite consultar, controlar y administrar los servicios (*units*), así como también gestionar el estado general del sistema operativo.

sudo useradd juan se crea un nuevo usuario /home/juan

sudo passwd juan Se crea una contraseña para el usuario juan

sudo usermod -l jos jaz se cambia el nombre jaz a jos

sudo usermod -d /home/nueva_carpeta jaz se cambia de directorio

sudo usermod -aG sudo jaz se le otorga permisos de administrador a jaz y se agrega al grupo sudo

sudo usermod -L jaz se bloquea la cuenta del usuario jaz

sudo userdel -r juan elimina el usuario Juan del sistema y también de /home

sudo groupadd profesores crea el grupo estudiantes en el sistema

sudo groupdel profesores elimina el grupo estudiantes en el sistema

sudo chown maria informe.txt cambia el propietario de informe.txt por maria

sudo shutdown -h now apaga el sistema de forma segura

sudo shutdown -r now apaga y reinicia el sistema de forma segura

reboot reinicia el sistema

su cambia al usuario root (administrador)

su juan cambia al usuario juan

who muestra los usuarios conectados actualmente en el sistema

whoami muestra el usuario actual que ejecuta el comando a diferencia de who que muestra todos los usuarios conectados actualmente en el sistema

`whoami` muestra el usuario actual dentro de otro comando como echo

echo "Su usuario es: `whoami`" el resultado de whoami es parte del resultado de echo

passwd juan cambia la contraseña del usuario juan (solo como root)

tar -cvf backup.tar carpeta/ empaqueta la carpeta "carpeta/" dentro de un archivo llamado **backup.tar**,

tar -czvf misLogs.tar.gz /tmp/logs empaqueta y comprime /tmp/logs en el archivo misLogs.tar.gz

tar -xvf misLogs.tar para desempaquetar el archivo misLogs.tar en directorio actual

tar -xvf misLogs.tar -C /home/usuario desempaqueta el archivo misLogs.tar en el directorio /home/usuario

tar -xzvf misLogs.tar.gz -C /home/usuario Desempaqueta y descomprime el archivo misLogs.tar.gz en el directorio /home/usuario

gzip informe.txt para comprimir, crea el archivo informe.txt.gz y elimina el informe.txt

zgrep "error" logs.gz busca la palabra error dentro del archivo comprimido log

wc comando para analizar la longitud y tamaño de un archivo

wc archivo.txt muestra n líneas, n palabras, n caracteres que hay en el archivo

wc -l devuelve la cantidad de líneas de un archivo

wc -w devuelve la cantidad de palabras(caracteres delimitados por espacios) que hay en una línea. Eje: echo "hola mundo" | wc -w ⇒ 2

fork() llamada del sistema que crea otro proceso (hijo) idéntico al actual, **hereda** (código, datos y variables, stdout, stdin, la prioridad, directorio actual) **y no hereda** (PID, contadores y estados del kernel propio). Retorna al padre(PID del hijo o -1) y retorna al hijo(valor 0). Ambos procesos padre e hijo se ejecutan en paralelo después del fork().

Usualmente después del fork() el proceso hijo ejecuta (**exec()**, **execv()**, **exec1()**, **execvp()**) para reemplazar su código y datos por otro programas. De esta manera el proceso Padre continúa su ejecución y el proceso hijo ejecuta un nuevo programa Por último ambos procesos terminan **exit()**.

wait() proceso padre espera finalización del proceso hijo

Exit() vaciar el buffer(stdout, stderr), imprime lo que está en el buffer, notifica al kernel que el proceso terminó termina el proceso liberando sus recursos

Exit(0) le indica al SO que el proceso terminó bien

ExecV("ls") se intenta reemplazar la imagen del proceso(código, datos,etc) por la del nuevo programa ls. Si la ejecución tiene éxito, se reemplaza y no se retorna nada y no se continúa con las siguientes líneas de código, se muestra un listado de archivos del directorio actual. Si la ejecución falla se retorna -1 y continúa con el código.

COMBINACIONES DE COMANDOS

ps -ef | grep "apa" | wc -l >> /tmp/cuenta se muestra los procesos que hay en el sistema y sirve como entrada para grep, luego filtra los procesos que contengan "apa" y sirve como entrada para wc, luego cuenta las líneas obtenidas y con >> redirige la salida hacia el archivo cuenta y lo guarda sin borrar lo anterior

ETAPAS PARA INICIAR EL SO HASTA EL LOGIN DEL USUARIO

Se enciende la computadora, el procesador ejecuta automáticamente el firmware BIOS/UEFI.

Para **BIOS** se busca en el cilindro 0, cabeza 0 y sector 1 del disco y se carga en memoria el MBC del MBR que contiene un código para encontrar el gestor de arranque GRUB en otra partición del disco y cargarlo en memoria.

Para **UEFI** se usa una partición especial EFI donde está el GRUB. Luego busca y carga el kernel que está en el directorio /boot/vmlinuz y la imagen /boot/initramfs, se inicializa el sistema y el kernel monta el sistema de archivo raíz (/) y ejecuta el primer proceso del sistema INIT (sbin/init) o SYSTEMD (lib/systemd/systemd) y arranca los servicios y el entorno del usuario

El proceso de bootstrap comienza con la ejecución del firmware (BIOS/UEFI), continúa con la carga del gestor de arranque (GRUB), luego el kernel del sistema operativo, y finaliza con la inicialización de los servicios del sistema y el inicio del entorno de usuario.

MBR (master boot record o registro de arranque maestro)

Tamaño MBR=512 bytes=tamaño estándar de sector

MBC (0-445), TABLA (446-509), FIRMA (510- 511)

Es un sector especial del disco, cilindro 0, cabeza 0, sector 1

MBC(master boot code, 446 bytes): Es un pequeño código que permite arrancar el SO.

Tabla de particiones(64 bytes): Indica como esta conformado el disco, contiene información de las particiones en el disco.

La MBR maneja hasta 4 particiones primarias (16 bytes cada una), si se necesitan más de 4 entonces, se manejan 3 particiones primarias y una extendida donde dentro pueden existir muchas particiones lógicas.

Los 2 bytes libre corresponden a la firma de arranque

GPT(UEFI): Maneja hasta 128 particiones por defecto y no tiene particiones extendidas.

Estructura Básica del SO

Cuando hay una distribución y está un SO (Win o LINUX) Hay muchas herramientas:

- 1) Paquete de software que permite diferenciar una distribución de otra.
- 2) Editores de texto (vi, emacs, joe)
- 3) Herramientas de networking (wireshark, tcpdump)
- 4) paquetes de oficina (LibreOffice)
- 5) Interface gráficas (GNOME/CINNAMON, KDE, LXDE)

RUNLEVELS (0-6)

Los **runlevels** son los distintos niveles de ejecución del sistema GNU/Linux que determinan qué servicios y procesos se inician. Se utilizan para definir modos de funcionamiento (monousuario, multiusuario, gráfico, apagado, reinicio, etc.).

- **0**: Apagado
- **1**: Modo monousuario
- **3**: Modo multiusuario sin interfaz gráfica
- **5**: Modo multiusuario con interfaz gráfica
- **6**: Reinicio

PROCESO INIT para sysVinit (versión vieja)

Es el primer proceso que se ejecuta al iniciar el sistema, en la ejecución del mismo se abre el archivo ubicado en /etc/inittab para determinar cómo inicializar el sistema y que servicios lanzar automáticamente. El archivo inittab contiene información del runlevel donde debe ejecutarse, el comando que debe ejecutar, el comportamiento que debe tener.

PROCESO systemd para systemd (2010)

Es el primer proceso que se ejecuta al iniciar el sistema, usa units(.service, .socket, .target) en lugar de scripts tradicionales. Reemplaza el concepto de RUNLEVELS por target (multi-user.target, graphical.target, etc.)

Mas definiciones de la Parte de Procesos:

Niveles de planificación: Permiten al kernel controlar la carga de trabajo, el uso de CPU, memoria del sistema

A corto plazo: Selecciona un proceso que obtendrá CPU entre la cola de listos, es decir que pase de estado "listo" a "ejecutando". Implementa los algoritmos FCFS, SJF, SRTF, PRIORIDADES, RR, etc. Si el algoritmo es apropiativo decide cuando expulsar un proceso.

A mediano plazo: Suspende, reanuda procesos. Decide qué procesos activos en memoria RAM deben suspenderse y guardarse en disco (swap out). Decide cuando volverlos a cargar (swap in). Actúa cuando hay poca memoria o cuando hay cambios de prioridad.

A largo plazo: Decide qué procesos del conjunto de trabajos pasan a memoria principal para su ejecución. Utilizado cuando hay procesos grandes para evitar sobrecarga del sistema.

Dispatcher: Es un módulo que utiliza la selección del proceso a ejecutar por el planificador de corto plazo. Realiza el cambio de contexto. **Transfiere el control de la CPU al nuevo proceso elegido.** Se cambia a modo usuario y luego salta a la próxima instrucción de ese proceso.

Loader: Carga código y datos del programa desde disco y lo ubica en memoria RAM para crear un proceso nuevo asociado al programa. Reserva espacio para las estructuras del proceso(stack, heap, datos, código, etc). Inicializa el PCB, cambia el estado del proceso a "listo" y así sea seleccionado por el planificador a corto plazo. Permite que un programa(archivo ejecutable) se convierta en proceso.

El SO referido al proceso tiene 3 tipos de colas de procesos (ready, bloqueados, nuevos, terminados) cola de new: procesos que acaban de crearse, Aún no están listos para ejecutarse (el SO está asignando recursos, PCB, memoria, etc.). Una vez que están listos, pasan a la **cola de listos**.

cola de listos (READY QUEUE): Procesos que **pueden usar la CPU**, pero están esperando turno. Manejada por el planificador, contiene información de prioridad, tiempo de espera, etc

Cola de bloqueados (waiting I/O queue): Procesos que **no pueden continuar** hasta que ocurra un evento externo. Esperan E/S (lectura de disco, teclado, red, etc.). Esperan un semáforo o señal. Cuando el evento ocurre el kernel los cambia a la cola de new

Cola de terminados (Exit, zombie): Procesos que ya terminaron su ejecución. El sistema mantiene su PCB un corto tiempo hasta que el padre lea su estado. Luego el kernel libera su PCB y recursos.

Proceso: Es una entidad abstracta que representa un programa en ejecución junto con su contexto de ejecución(código, datos, pila, registros y PCB)

En GNU/Linux, los procesos son **administrados por el kernel**, que se encarga de planificarlos, suspenderlos, reanudarlos o finalizarlos.

(RECORDANDO): El SO tiene como servicio: Administración y planificación del procesador, Administración de memoria(un proceso es cargado en memoria principal para poder ejecutarlo, cuando termina la ejecución se libera esa memoria para otro proceso), Administración de almacenamiento, Administración de dispositivos, Detección de errores y respuestas (ejemplo: Si un programa quiere acceder a un lugar que no le corresponde de la memoria, el SO tiene que detectar y hacer que no afecte al resto de los programas). Interacción del usuario(shell), Contabilidad (Un SO no solo administra los procesos, el hardware, los recursos, la memoria, los discos sino además hace contabilidad de todo el uso. Mantiene cuánto tiempo lleva ejecutando un proceso. Cuanta memoria ocupa, cuánto ocupa. Cuanta entradas/salidas utiliza)

PSW: Es un registro especial de la CPU que almacena el estado actual de la CPU y del programa que se está ejecutando

- bit de modo ⇒ modo usuario o kernel
- bits de condición ⇒ resultados de operaciones (carry, overflow, cero, etc)
- bits de interrupción ⇒ indica si las interrupciones están o no habilitadas
- bits de control del CPU ⇒ configuran ciertas opciones internas del CPU
- prioridad del proceso ⇒ guarda prioridad del proceso en ejecución

PCB: Es una estructura de datos del kernel que contiene toda la información necesaria para describir y gestionar un proceso. Es lo primero que se crea cuando se crea un proceso y es lo último que se borra cuando termina. En paralelo se crea el proceso, se crea el PCB y se carga el programa en memoria. El PCB creado se almacena en la memoria RAM dentro de la zona protegida del kernel y está en la tabla de procesos.

PLANIFICADOR: Es el componente del kernel que tiene un grupo de procesos en estado de "ready" y decide que proceso va a usar la CPU y cuando. Usa el atributo de prioridad para decidir. Maneja colas de planificación

QUANTUM: Es el tiempo asignado por el SO a cada proceso para usar la CPU de forma continua antes de que ocurra una interrupción de reloj y el SO decida si continuar o pasar a otro proceso

Herencia de un proceso creado: todo proceso hereda 3 archivos abiertos, stdin, stdout, stderr

Un proceso tiene componentes como:

Sección de código del programa: Instrucciones cargadas en memoria principal

Sección de datos: Variables globales, estáticas

Heap: memoria dinámica

Stack: Se almacena variables locales, pasajes de parámetros, direcciones de retorno de funciones, PSW, registros de la CPU. Tiene una pila para el modo usuario y otra pila para el modo kernel.

Registros de CPU: tienen resultados intermedios, direcciones, contadores. Se almacena en la Stack del proceso si ocurre un cambio de modo, se almacena registros generales y especiales en el PCB si ocurre un cambio de contexto.

PSW(program status word): contiene el bit de modo. Se almacena en la Stack del proceso si ocurre un cambio de modo, se almacena en el PCB si ocurre un cambio de contexto.

PCB(bloque de control de proceso): Es el contexto del proceso.

- contiene PID(id del proceso),
- PPID(id del proceso padre),
- UID/ GID (identificador de usuario o grupo que lo creó)
- estado actual
- registros de la CPU (generales, especiales, PC)
- prioridad
- espacio de dirección asignado al proceso (el SO le asigna ese espacio y la CPU controla ese espacio)
- información de planificación (quantum restante)
- gestión de memoria (tabla de páginas o segmentos, límite de memoria asignada)
- recursos usados

Cambio de contexto: Se produce cuando la CPU cambia de un proceso a otro

- 1) Se resguarda el contexto del proceso saliente, se cambia el estado de ese proceso en espera y luego la CPU continuará con ese contexto
- 2) Se carga el nuevo contexto de proceso, cambia su estado y comienza a ejecutar desde donde quedo.

Procesos swapeados: Son procesos que se trasladan temporalmente desde la RAM hasta el área del disco llamada swap. Pero su PCB reside en la memoria RAM y tiene su estado "swapeado".

De "espera swapeado" a "listo swapeado": La PCB correspondiente al proceso cambia su estado, se pasa a la cola de "ready swapeado". Pero el espacio de direcciones sigue en la swapp. Cuando cambie de estado "**ready swapeado**" a "**ready**" entonces se trae su espacio de direcciones a RAM

Procesos CPU Bound: Son procesos que consumen mucho tiempo de CPU. Ejemplo: compilador

Procesos I/O Bound: Son procesos que esperan muchas operaciones de dispositivos externos.

Proceso background

Un proceso en **background** es aquel que **se ejecuta sin ocupar la terminal actual**, permitiendo al usuario **seguir utilizando la consola para otros comandos..** Para hacerlo se agrega un & al final del comando

Proceso foreground

Cuando un proceso se ejecuta en **foreground**, significa que **ocupa la terminal actual y requiere la interacción directa del usuario.**

Mientras el proceso se está ejecutando, la **línea de comandos queda bloqueada:** el usuario no puede escribir otros comandos hasta que el proceso termine.

Procesos con starvation(inanición): Son procesos que nunca se ejecutan debido a que los procesos de mayor prioridad son seleccionados primero. En colas multinivel puede pasar si la cola de prioridad alta no se desocupa.

Procesos interactivos: requieren interacción con el usuario (navegadores,editores, etc)

Procesos Batch: Se ejecutan en segundo plano cuando el SO tiene recursos libres, tienen baja prioridad, no hay interacción con el usuario. Algoritmos de planificación no apropiativos usan estos tipos de procesos.

Cola Multinivel: El SO mantiene varias colas en estado ready, cada una tiene su algoritmo de planificación. Se va a tener una cola para procesos interactivos y otra cola para procesos batch. Los procesos no pueden pasar de una cola a otra. El planificador administrado por el SO(kernel), selecciona las colas por prioridad alta y cuando esa cola queda vacía pasa a la siguiente.

Colas multinivel con realimentación: Los procesos pueden pasar de una cola a otra para evitar inanición. El SO(kernel) administra los procesos que ocupan demasiado CPU y los mueve a una cola de menor prioridad y los procesos que esperan mucho pueden ser promovidos. Cambiando la prioridad del proceso y cambiando el comportamiento.

Técnica de envejecimiento (aging): Para evitar procesos de inanición se usa en los algoritmos de planificación con prioridad fija, consiste en aumentar gradualmente la prioridad de procesos que han esperado mucho tiempo

Procesos No apropiativos(no se interrumpen): FCFS - SJF - prioridades fijas. no utilizan la técnica aging porque no manejan prioridades dinámicas y no provocan inanición prolongada

Procesos apropiativos(se interrumpen): RR - prioridades fijas- SRTF

Algoritmos de Planificación:

VRR(virtual round robin): El proceso que regresa de una operación I/O se coloca en una cola auxiliar con su quantum restante. Los procesos en esta cola auxiliar tienen más prioridad que los otros. En su turno completan su rafa de CPU anterior.

Definiciones acerca de MEMORIA:

Dirección virtual o lógica: Hace referencia dentro del proceso. Ejemplo: Un proceso ocupa 2000 bytes entonces desde la direcciones 0 hasta 2000 bytes son virtuales

Dirección física: Hace referencia a la memoria RAM del sistema.

Particiones fijas: El disco se divide por particiones(tamaños iguales o diferentes) que representan un proceso. Puede tener particiones sin procesos. Su espacio está predefinido

Particiones dinámicas: Las particiones se ajustan de acuerdo al requerimiento del proceso.

EJEMPLO: el proceso ocupa 20, requiere memoria, se crea la partición de 20, el proceso deja su ejecución, se libera esa partición y si hay otra partición contigua se unen

PROBLEMA, hay espacio libre entre las particiones creadas y no pueden ser ocupadas por ningún proceso.

Hay Fragmentación externa.

Existen algoritmos para PARTICIONES DINÁMICAS :

Primer ajuste: Se tiene una lista de particiones y de acuerdo al requerimiento del proceso se recorre la lista y en la primera partición cuyo requerimiento de memoria por el proceso sea menor o igual que la partición entonces se le asigna la partición. EJEMPLO: el proceso ocupa 20 y la primera partición ocupa 50 entonces se almacena en la primera partición de la lista ya que $20 < 50$

Mejor ajuste: Busca la partición que mejor se ajuste al requerimiento del proceso. EJEMPLO: el proceso ocupa 20 y hay 2 particiones una de 30 y otra de 50 entonces el proceso se almacena en la partición de 30

Peor ajuste: Es lo contrario de mejor ajuste. El proceso se almacena en la partición de 50

Próximo ajuste:

PROBLEMA, hay mucho espacio libre por cada partición si el proceso ocupa poco. Hay fragmentación interna (espacio libre de la partición que no puede ser ocupada por otro proceso)

Tanto en **particiones fijas** como en **dinámicas**, cada proceso ocupa **una región contigua** de la memoria. Por lo tanto, **todas las direcciones lógicas** del proceso se mapean de forma **lineal** sobre esa región.

Fragmentación interna: Se produce en particiones fijas, es la porción de la localidad que queda sin asignar

Fragmentación externa: Se produce en particiones dinámicas. Son huecos que quedan en la memoria a medida que los procesos finalizan.

SOLUCIÓN (costosa) $\Rightarrow$ Realizar una compactación para tener esos huecos en forma contigua en memoria y tener memoria libre para almacenar un proceso

Paginación: Es un método de administración de memoria que divide la memoria RAM (Frames) y la memoria lógica/ espacio de direcciones (páginas) en bloques de igual tamaño para poder asignar los procesos de manera flexible y eficiente. No se produce fragmentación externa y sólo se genera fragmentación interna en el último marco.

El SO (el kernel) asigna y administra una tabla de páginas para cada proceso donde en cada entrada de la tabla se guarda (nro de página, nro de frame, bit de modificación, referencia, protección)

El SO (el Kernel) crea, mantiene, destruye estas tablas cuando los procesos se crean o se terminan

El SO (el Kernel) mantiene una tabla o bitmap de frames libres , donde cada entrada de esa tabla representa un frame que puede estar marcado como "disponible" o "asignado a una página"

El SO (el Kernel) configura un registro especial de la CPU para que la MMU sepa a qué dirección de tabla de páginas ir del proceso actual cuando hay un cambio de contexto

El SO (el Kernel) tiene un manejador de fallos de página cuando un proceso necesita acceder a una página que no está cargada en memoria (bit de validez=0). Donde localiza la página en disco (swapp), Reserva un frame libre (si no hay libres, se usa una página víctima), carga la página en RAM, actualiza la tabla de páginas del proceso, continua la ejecución.

El SO (el Kernel) actualiza la TLB en cambios de contexto o modificaciones en las tablas.



Frame o Marco: Bloque de tamaño fijo de memoria RAM igual al tamaño de una página

Página: Bloque de memoria lógica de un proceso.

Ejemplo y análisis: Un proceso está dividido en páginas, cada página tiene el mismo tamaño del marco de memoria, con lo cual referenciando un nro de página y un desplazamiento se tiene la dirección de almacenamiento real.

Proceso A de 5 KiB; Proceso B de 3 KiB; RAM de 8 KiB

```
RAM (8 KiB total)

Frame 0 → A0 (2 KiB)
Frame 1 → A1 (2 KiB)
Frame 2 → A2 (1 KiB usado + 1 KiB libre pero reservado)
Frame 3 → B0 (2 KiB)

→ B1 no entra; faltan frames
```

Frame 2: Hay fragmentación interna, cada frame representa 1 sola página de proceso

B1: debe ir a otro lugar como la memoria SWAPP

MMU Traduce direcciones lógicas a físicas, verifica si el proceso tiene permisos de lectura, escritura, ejecución a esa dirección

TLB: Es una memoria caché de la MMU, guarda traducciones recientes lógicas⇒frames. El kernel interactúa para invalidar.

TLB HIT la traducción de la dir. logica esta en la cache entonces no accede a la tabla de páginas, se obtiene el frame directamente

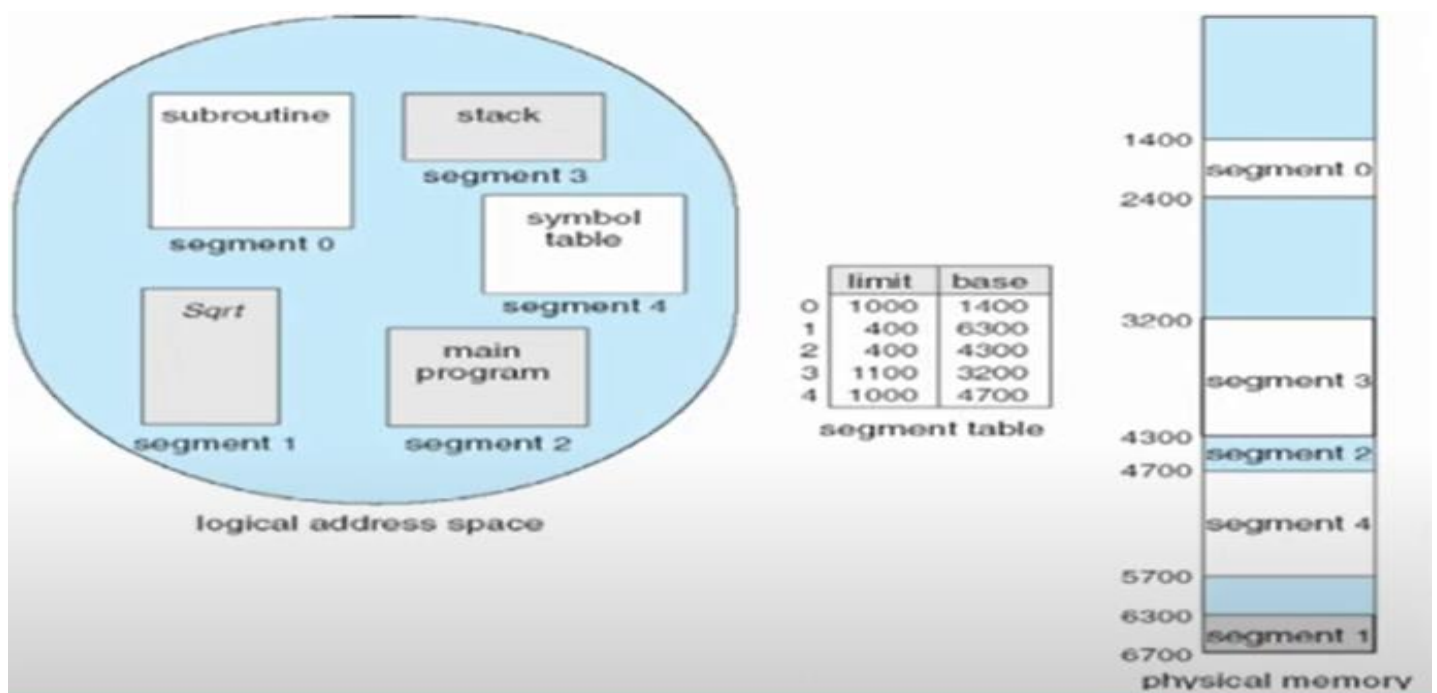
TLB MISS la traducción lógica no esta, entonces accede a la tabla de páginas para saber si está o no en la memoria RAM y obtener el frame correspondiente

Paginación con tabla de 1 nivel: Se tiene un proceso para un espacio de direcciones organizado en páginas y una tabla donde cada entrada se asocia una página lógica(usando nro de página) con un frame(usando nro de frame) de memoria física. La MMU usa el nro de página para obtener al nro de frame y calcular la dirección física.

Paginación con tabla de 2 niveles: Se tiene una tabla externa (contiene direcciones base de cada tabla interna)y varias tablas internas. Donde las tablas internas para las páginas necesarias están cargadas en RAM. Se accede primero a la tabla externa y luego a la interna = 2 accesos). Usa **TLB** (buffer para cachear traducciones). Por ejemplo: Ideal para un proceso que solo usa una pequeña parte del espacio de direcciones

Paginación con tabla invertida: Se tiene una sola tabla global para toda la memoria RAM. Cada entrada de la tabla representa un frame e indica qué proceso y que página lo ocupa.

Segmentación: Es un esquema de administración de memoria que divide el **espacio de direcciones de un proceso** en **segmentos lógicos de distinto tamaño**, en lugar de páginas fijas.



Paginación

- **Unidad de división:** Páginas (bloques fijos).
- **Tamaño:** Fijo (ej. 4 KiB).
- **Criterio de división:** Numérico / por cantidad de bytes.
- **Fragmentación:** Interna (en el último *frame*).
- **Tabla usada:** Tabla de páginas.
- **Traducción:** Página
- →right arrow
- → *Frame*.
- **Ventaja:** Evita fragmentación externa.
- **Desventaja:** No sigue la lógica del programa.

Segmentación

- **Unidad de división:** Segmentos (bloques variables).
- **Tamaño:** Variable.
- **Criterio de división:** Lógico / por significado del código.
- **Fragmentación:** Externa (segmentos de distintos tamaños).
- **Tabla usada:** Tabla de segmentos (base + límite).
- **Traducción:** Segmento
- →right arrow
- → Dirección base + desplazamiento.
- **Ventaja:** Representa mejor la estructura lógica del programa.
- **Desventaja:** Causa fragmentación externa.

El SO (el kernel) asigna memoria física a cada proceso y le informa a la CPU su espacio asignado. Mantiene la información de sus registros base y límite. El HW usa esos registros para controlar que no acceda a lugares indebidos de memoria

El SO (el kernel) asigna y administra una tabla de segmentos para cada proceso donde en cada entrada de la tabla se guarda (Dirección base, longitud, bit de control que son los permisos de lectura, escritura, ejecución)

El SO (el kernel) configura los registros especiales de la CPU que identifican el segmento(datos, código, pila,...) y otro reg que contiene la dirección base de la tabla de segmentos cuando hay un cambio de contexto

El SO (el kernel) debe administrar los espacios libres generados por la segmentación entonces realiza compactación si es necesario

Segmentación paginada: Es una técnica de administración de memoria que consiste en separar el espacio de direcciones de un proceso en segmentos y cada segmento está compuesto por páginas para evitar fragmentación externa. Puede tener fragmentación interna pero es mínima. El SO asigna y mantiene 2 tablas, una para el segmento y otra para indicar las páginas del segmento.

DIRECCIÓN LÓGICA = (Segmento, NRO de página, Desplazamiento dentro de la página)

DIRECCIÓN FÍSICA = (FRAME * tamaño de página) + desplazamiento

DIRECCIÓN FÍSICA = Dir.base de página + desplazamiento

Tabla de Segmentos		Tabla de Páginas		
Núm. Seg.	Dir. base	Nro. Segmento	Nro. Página	Direc. Base
1	500	1	1	40
			2	80
			3	60
2	1500	2	1	20
			2	25
			3	0
3	5000	3	1	120
			2	150

Dir. base Indica la dirección de la tabla de páginas de ese segmento.

Primeros algoritmos, Partición fija o dinámica, donde todo el proceso estaba cargado en memoria en direcciones contiguas, dentro de una partición, locación continua

Bit de modificación o dirty:

1 (página modificada)

0 (página no modificada).

Se encuentra en la entrada de la tabla

Bit de validez o bit de presencia:

1 (página se encuentra en RAM)

0 (fallo de página, no se encuentra en RAM y puede estar en SWAPP si ya fue usada antes o en memoria secundaria si nunca se usó).

Se encuentra en la entrada de cada página de la tabla de páginas del proceso, es un bit de control.

Página víctima: Es la página que el SO decide sacar de la memoria RAM cuando ocurre un fallo de memoria y no hay ningún frame libre disponible.

Fallo de página (page fault): Sucede cuando una página que se intenta acceder no se encuentra en memoria principal (RAM).

Pasos:

La MMU consulta la tabla de páginas del proceso y la entrada indica "página ausente".

El bit de validez está en 0 (Página ausente, no está cargada en RAM, está en disco) Ocurre fallo de página, entonces, la **MMU(hw)** lanza una interrupción y transfiere el control al SO. Se localiza la página en el área de SWAP (o archivo de paginación) del disco. Se copia la página desde el disco a un frame libre de la RAM. Se actualiza la tabla de páginas del proceso marcando el frame donde se cargo la página y cambiando el bit de validez en 1. El proceso continúa su ejecución desde la instrucción interrumpida.

Si no hay FRAMES libres entonces el SO selecciona una página víctima usando un algoritmo de reemplazo (FIFO, LRU, ÓPTIMO). Si esa página seleccionada fue modificada (bit dirty=1 o bit de modificación=1) se escribe en el disco para no perder los datos actualizados. Se carga la copia de la página desde el disco en ese Frame liberado. Se actualiza la tabla de páginas del proceso saliente (marca la página "no presente") y del proceso entrante (marca la página "presente") y cambiando el bit de validez en 1. El proceso continúa su ejecución desde la instrucción interrumpida.

SHELL SCRIPTING - Temas del 2DO Parcial

SHELL SCRIPT es un archivo de texto plano que contiene una serie de comandos que el shell (BASH, sh, zsh) ejecuta secuencialmente sin necesidad de una compilación. Para ejecutar la script en la shell se usa

./archivo.sh

Están orientados a automatizar tareas repetitivas:

1) Administración del sistema

- Backups
- monitoreo
- Limpieza de logs
- Creación de usuarios

2) Procesamiento de archivos

- búsqueda
- filtrado
- manipulación de texto

3) automatización general

- Instalación de software
- Despliegue de aplicaciones
- Pruebas automatizadas

4) Tareas simples donde no hace falta un lenguaje compilado

Los **SHELL SCRIPTS** no se compilan, no tienen etapa de compilación, se interpretan línea por línea por el shell sin necesidad de generar un binario de máquina por ese motivo son fáciles de escribir, modificar y ejecutar.

SHELL SCRIPTING son shell scripts o conjuntos de comandos que el SHELL ejecuta para automatizar tareas del SO UNIX/LINUX. Se clasifican como débilmente tipados ya que su conversión es implícita de tal manera que se pueden ejecutar con un texto + numero sin errores.

Formas de concatenar STRINGS

```
nombre="archivo"
extension=".txt"
completo="${nombre}${extension}"
# Resultado: archivo.txt
```

```
ruta="/home/usuario"
ruta+="/descargas"
# Resultado: /home/usuario/descargas
```

Comandos:

echo muestra texto o el valor de una variable

echo \$HOME muestra la ruta del directorio personal del usuario actual

echo "Hola mundo" muestra el texto en pantalla

echo "Tu nombre es \$nombre" concatena texto + variable nombre

echo -n "Hola mundo" muestra el texto en la misma línea

echo "\$n1 + \$n2 = \$(expr \$n1 + \$n2)" $\Rightarrow 2 + 2 = 4$

echo "\$n1 x \$n2 = \$(expr \$n1 * \$n2)" $\Rightarrow 2 * 2 = 4$

read lee datos en pantalla y los guarda en una variable

read nombre guarda lo que se escribe en la variable nombre

read -p "Ingresa tu nombre: " nombre muestra texto y guarda lo que se escribe en la variable nombre

para comentar en una script

nano crea un archivo de texto y permite escribir contenido a diferencia con Touch que solo permite crear

nano mostrar.h crea un archivo mostrar.h

./mostrar.sh o **./mostrar** ejecuta el archivo script llamado mostrar

cd ~ te dirige al directorio personal

cd \$home te dirige al directorio personal

cd te dirige al directorio personal

Exit N termina la ejecución de inmediato de la SCRIPT y devuelve el código de salida N al SO (codigo que recibe el shell, otros programas o un script padre) y N Puede ser 0..255(8 bits de código que utiliza UNIX)

Exit 0 ejecución correcta

Exit 1 error genérico

Exit 2 error de uso incorrecto de shell o de los argumentos

Exit 126 comando encontrado pero no ejecutable

Exit 127 comando no encontrado

Exit 130 script terminada con CTRL+ C

\$? contiene el código de salida del último comando ejecutado(0: verdadero o 1:falso)

Cut extrae columnas de texto de un archivo (d:delimitador, f:columnas a extraer, c.caracteres a extraer)

cut -d: -f1 /etc/passwd extrae la primera columna del archivo passwd delimitada por ":"

cut -d"," -f1,3 archivo.csv extraer la columna 1,3 del archivo.csv delimitado por ","

cut -c1-5 archivo.txt extrae los 5 primeros caracteres de la primera línea de un archivo

tr transforma o elimina caracteres(-d) o espacios(-s)

echo "hola" | tr 'a-zA-Z' la salida de primero es la entrada del segundo entonces transforma el texto "hola" a mayúscula HOLA

echo "hola" | tr 'aA' salida esperada holA

echo "a1b2c3" | tr -d 0-9 salida esperada abc

echo "hola mundo" | tr -s " " salida esperada hola mundo

sleep N pausa la ejecución del script por N segundos y luego continua su ejecución con la siguiente línea

Variantes: sleep 1h, sleep 1m, sleep 1h 1m, sleep 1.5 m

basename sirva para obtener el nombre de un archivo

nombre=\$(basename "\$archivo")

Getent se utiliza para consultar la base de datos administrativa del sistema, consulta los archivos locales y los archivos remotos (red)

getent etc/passwd "\$usuario" busca en la base de datos del sistema el archivo passwd si existe un usuario que está contenido en la variable en caso que exista devuelve el nombre del usuario y lo muestra por pantalla

Groups muestra los nombres de grupos (primario y secundarios) de un usuario específico. su sintaxis es nom_usuario: grupo1 grupo 2 grupo 3.

Al ejecutar solo **groups** muestra los grupos del usuario actual que ejecuta la script.

groups nom_usuario muestra los grupos de nom_usuario

Combinaciones de comandos:

who | grep -w "\$usuario" >/dev/null la lista de usuarios conectados en el sistema(who) es la entrada para buscar la linea(grep -w) donde aparece exactamente el \$usuario pero como solo importa la búsqueda y el código de retorno \$? se descarta la salida en /dev/null. código de retorno es igual a 0

arch=\$(find /home -type f -name "*.doc" 2>/dev/null) Se obtienen archivos regulares .doc del directorio /home y se eliminan error adicionales

Declaración y asignación de variables:

nombre="Juan" para acceder a su contenido es \$nombre o tambien \${nombre}

edad=20 para acceder a su contenido es \$edad o tambien \${edad}

Parámetros posicionales:

- **\$1, \$2, \$3,...,\$n**
- **\$0** contiene el nombre del script que se está ejecutando
- **\$#** indica la cantidad de argumentos
- **\$@** indica la lista de argumentos preservando separación
- **\$*** indica la lista de argumentos como un único string
- **\$?** indica código del retorno del último comando ejecutado

Forma prolija de usar los parámetros en una SCRIPT

Expresion: nom.sh 2 3, a la script nom se le pasan 2 argumentos

{ n1= \$1 se recibe el parametro1 y se guarda en n1 dentro de la script

n2= \$2 se recibe el parametro2 y se guarda en n2 dentro de la script

}

Estructuras de control

Select muestra todas las opciones por pantalla y espera una elección del usuario y después con case se maneja las distintas opciones elegidas.

```

case $variable in
"valor 1")
    bloque
    ;;
"valor 2")
    bloque
    ;;
*)
    bloque
    ;;
esac

```

```

echo "1) Listar"
echo "2) DondeEstoy"
echo "3) QuienEsta"
read opcion

case $opcion in
    1) ls ;;
    2) pwd ;;
    3) who ;;
    *) echo "Opcion incorrecta" ;;
esac

```

```

select accion in Nuevo Salir
do
    case $accion in
        "Nuevo")
            echo "Seleccionado: Nuevo"
            ;;
        "Salir")
            exit 0
            ;;
    esac
done

```

Select + Case y sale con Exit 0

```

select opcion in lista
do
    comandos
done

```

Select sin Case y sale con Exit o Break

```

num=(1 2 3)

for i in "${num[@]}"; do
    echo "Elemento: $i"
done

```

num es un ARRAY

Salida:

Elemento: 1
Elemento: 2
Elemento: 3

```
for x in "${num[*]}"; do echo "$x"; done
```

Salida:

1 2 3

```
for ele in "$num"; do
```

Salida:

1

```
for a in "$1"/*; do
    if [ -f "$a" ]; then
        if [ -r "$a" ] && [ -w "$a" ]; then
            ((cantidad++))
        fi
    fi
done
```

Salida:

cantidad de archivos regulares con permisos de lectura y escritura a partir del directorio que tiene \$1 y correspondiente al usuario que ejecuta el SCRIPT

```
for i in "${!archivos[@]}"; do
    echo "${archivos[$i]}"
done
```

Salida:

índices del arreglo archivos 0 1 2 3 .. etc

```
# Recorrer archivos del directorio actual
for i in "$PWD"/*; do
    if [ -f "$i" ] && [ -x "$i" ]; then
        mv "$i" "$HOME/bin/"
        echo "Movido: $(basename "$i")"
        ((cantidad++))
    fi
done
```

Cada archivo ejecutable del HOME del usuario actual se mueve al subdirectorio /bin gracias a la ruta de cada archivo que se guarda en \$i. Además se muestran los nombres de los archivos y se contabiliza cada uno.

```
for i in 1 2 3; do
    if [ $i -eq 2 ]; then
        break # sale del bucle
    fi
    echo $i
done
```

Salida: 1

```
for i in 1 2 3; do
    if [ $i -eq 2 ]; then
        continue # se salta el 2
    fi
    echo $i
done
```

Salida: 1 3

OPERACIONES CON ARREGLOS:

nombres=("pepe" "Ana") o **números=(2 2 3)** agregar valores a un array
número=(1 2 3) crear array con NÚMEROS
echo "\${nombres[0]}" mostrar el primer elemento
echo "\${nombres[@]}" "mostrar todos los elementos en la misma línea
echo "\${nombres[@]}" " con **FOR**, muestra todos los elementos en distintas líneas
echo \${#arreglo[@]} o **\${#arreglo[*]}** obtener el tamaño del arreglo
unset arreglo[2] eliminar elemento del arreglo
echo "\${array[\${expr \${#array[@]} - 1}]}" muestra el último elemento del array
unset array[\${expr \${#array[@]} - 1}] elimina el último elemento del array, Forma antigua
echo "\${!archivos[@]}" muestra los índices del arreglo

push

1) **push()** { pila+=("\$1") }
 2) **push_alt()** { pila[\${#pila[@]}]=\$1 }

pop

1) **unset** pila[\${#pila[@]} - 1]]
 2) **unset** pila[\${expr \${#pila[@]} - 1}]

```
while [ condicion ] # Mientras se cumpla la condición
do
    bloque
done
```

FUNCION:

```
miFuncion() {
    echo "Parametro 1: $1"
    echo "Parametro 2: $2"
}
```

Declaración de una función

En la SCRIPT: **miFuncion** "Hola" "Mundo"

Salida:

Parametro 1: Hola
 Parámetro 2: Mundo

Notas:

1. El **return** en una función devuelve valores numéricos de 0-255 y no devuelve un arreglo por ejemplo "\${arreglo[@]}". Entonces se usa el comando **echo "\${arreglo[@]}"**

- Operaciones aritméticas:

Permite realizar cálculos numéricos.

+	Suma	expr 4 + 5 → 9
-	Resta	expr 10 - 3 → 7
*	Multiplicación	expr 4 * 5 → 20
/	División entera	expr 7 / 2 → 3
%	Módulo	expr 10 % 3 → 1

- Operaciones relacionales:

Usadas para comparar valores (retornan 1 = verdadero, 0 = falso).

=	Igual	expr 5 = 5 → 1
!=	Distinto	expr 5 != 3 → 1
>	Mayor que	expr 7 > 2 → 1
<	Menor que	expr 2 < 1 → 0
>=	Mayor o igual	expr 3 >= 3 → 1
<=	Menor o igual	expr 1 <= 5 → 1

- Operaciones lógicas:

Usadas para combinar condiciones.

·	·	OR (devuelve el primer valor no nulo)	
&		AND (devuelve 0 si alguno es 0)	expr 5 \& 0 → 0

- Operaciones con cadenas:

Permite trabajar con texto.

• Comparación de cadenas

Funciona igual que los relacionales:

expr "hola" = "hola"

• Longitud de una cadena

expr length "texto" → 5

• Substring (extracto)

expr substr "computadora" 2 4 → "ompu"

(Posición inicial: 1)

Operaciones aritméticas

- `+`
- `-`
- `*`
- `/`
- `%`

Comparaciones numéricas

- `>`
- `<`
- `>=`
- `<=`
- `=`
- `!=`

Comparaciones de strings

- `expr length "cadena"`
- `expr substr "cadena" inicio largo`

Mostrar operaciones aritméticas

`echo "$(expr $n1 + $n2)"` muestra la suma de 2 números

`echo "$(expr $n1 - $n2)"` muestra la resta de 2 números

`echo "$(expr $n1 \* $n2)"` muestra la multiplicación de 2 números

`echo "$(expr $n1 / $n2)"` muestra la división entera de 2 números

En SHELL SCRIPT no existen los decimales

TEST expresión o [expresión] usada en la PRÁCTICA

1) Evaluación de archivos

- `-f archivo` → archivo regular
- `-d archivo` → directorio
- `-r archivo` → lectura
- `-w archivo` → escritura
- `-x archivo` → ejecución
- `-e archivo` → existe

2) Evaluación de cadenas

- `-z cadena` → cadena vacía
- `-n cadena` → cadena NO vacía
- `cadena1 = cadena2`
- `cadena1 != cadena2`

3) Evaluación numérica

- `n1 -eq n2`
- `n1 -ne n2`
- `n1 -lt n2`
- `n1 -le n2`
- `n1 -gt n2`
- `n1 -ge n2`

<code>-eq</code>	Igual que
<code>-ne</code>	Distinto
<code>-gt</code>	Mayor que
<code>-lt</code>	Menor que
<code>-ge</code>	Mayor o igual que
<code>-le</code>	Menor o igual que

Comparación entre archivos:

<code>archivo1 -nt archivo2</code>	archivo1 es más nuevo
<code>archivo1 -ot archivo2</code>	archivo1 es más viejo
<code>archivo1 -ef archivo2</code>	Son el mismo archivo

El resultado será:

- 0 (true)** si la expresión es verdadera.
- 1 (false)** si es falsa.

- **En la evaluación de archivos** se debe usar las comillas dobles "" para evitar la división cuando la variable contiene espacios o está vacía.

Ejemplo 1 de SHELL SCRIPT

```
# cargar usuarios del grupo "users"
linea=$(getent group users | cut -d: -f4)

IFS=',' read -r -a usuarios <<< "$linea"

case "$1" in
    -b)
        n=$2
        if [ "$n" -lt "${#usuarios[@]}" ]; then
            echo "${usuarios[$n]}"
        else
            echo "Error: posición inexistente"
        fi
        ;;
    -l)
        echo "${#usuarios[@]}"
        ;;
    -i)
        for u in "${usuarios[@]"; do
            echo "$u"
        done
        ;;
    *)
        echo "Uso: $0 {-b n | -l | -i}"
        ;;
esac
```

Lo que hace: Filtra usuarios del sistema pertenecientes al grupo users, realiza 3 operaciones

Ejemplo 3 de SCRIPT

```

if [ $# -ne 1 ]; then
    echo "Pasar como mínimo un parametro"
    exit 1
fi

if [ ! -d "$directorio" ]; then
    echo "El directorio no existe"
    exit 4
fi

cantidad=0

for a in "$1"/*; do
    if [ -f "$a" ]; then
        if [ -r "$a" ] && [ -w "$a" ]; then
            ((cantidad++))
        fi
    fi
done

echo "Cantidad de archivos con permiso de lectura y escritura: $cantidad"

```

Lo que hace: Se pasa como parámetro un directorio y se verifica si se recibe un parámetro, verifica si se trata de un directorio. Por último muestra la cantidad de archivos del directorio donde el usuario que ejecuta tenga solo permisos de lectura y escritura, sin contar los subdirectorios.

Parte de ADMINISTRACIÓN DE ARCHIVOS:

Cluster es un conjuntos de sectores consecutivos que el SO utiliza como unidad lógica de almacenamiento más pequeña de archivos

i-nodo en UNIX System V es una estructura del sistema de archivos, que representa un archivo o un directorio. No contiene datos del archivo pero sí metadatos:

Atributos / Metadatos de un Inodo

- **Tipo de archivo**
- **Permisos**
- **UID / GID** (Identificador de Usuario / Identificador de Grupo)
- **Tamaño**
- **Punteros a bloques**
- **Contador de links** (*link count*)
- **Timestamps:**
 - **atime** → último acceso
 - **mtime** → última modificación de contenido
 - **ctime** → último cambio de **metadatos del inodo**

atime: última vez que se leyó o accedió al archivo

mtime: última vez que se modificó el contenido del archivo(editar, guardar)

ctime: última vez que cambiaron los metadatos(permisos, propietario..)

Para un directorio(archivo especial):

Compuesto por 13 direcciones a bloques de datos:

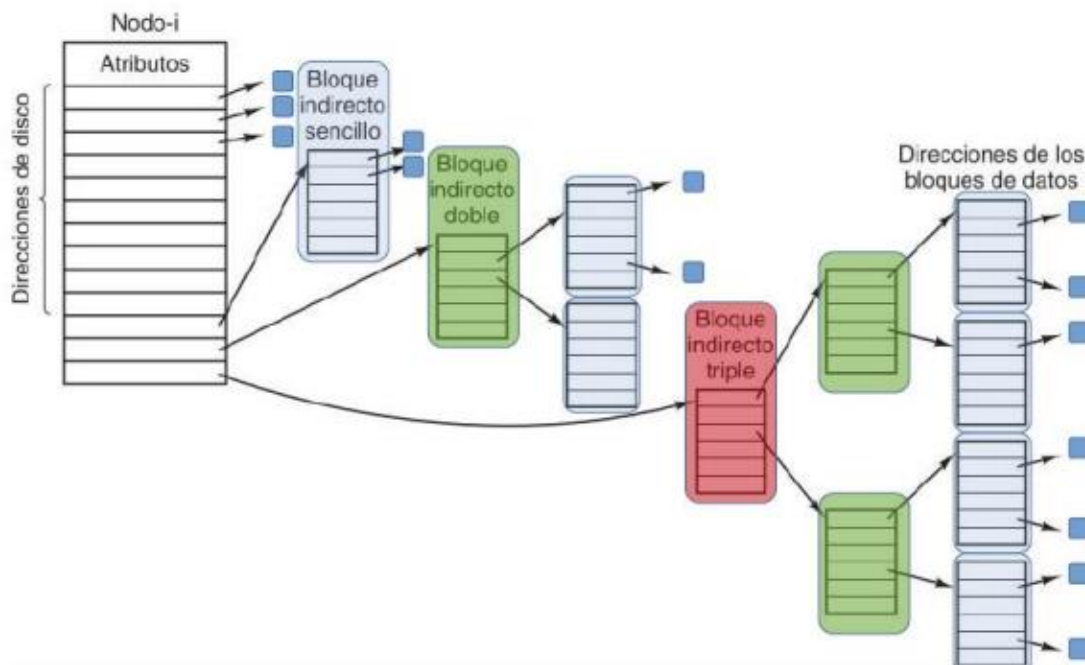
10 punteros directos ⇒ señalan directamente un bloque de datos

1 puntero indirecto simple ⇒ apunta a un bloque que tienen punteros a bloques de datos

1 puntero indirecto doble ⇒ apunta a un bloque que tiene punteros a bloques indirectos simples

1 puntero indirecto triple ⇒ apunta a un bloque que tiene punteros a bloques indirectos dobles
(el límite de tamaño está dado por los niveles de indirecto)

(a) Realizar un gráfico que represente la estructura del I-NODO y de los bloques de datos. Cada cluster es de 1 Kib (Kibibits).



Métodos para la gestión de espacios libres

Tabla de bits:

- Se mantiene un vector de bits, donde cada bit representa un bloque del disco: 0 es bloque libre y 1 es bloque ocupado.
- El bitmap suele mantenerse en memoria para consultas rápidas.
- Ventajas: muy fácil identificar bloques contiguos (útil para asignación contigua); ocupa poco espacio (1 bit por bloque); rápida búsqueda usando operaciones de bits.
- Desventajas: si el disco es grande, el bitmap puede ser, relativamente, extenso; puede requerir leer varias palabras de memoria para encontrar un rango libre.

Lista enlazada:

- Todos los bloques libres están encadenados uno tras otro formando una lista.
 - Cada bloque libre contiene un puntero al siguiente bloque libre.
 - El superblock mantiene la dirección del primer bloque libre.
 - Ventajas: sencilla de implementar; no requiere espacio en memoria más que un puntero al primer bloque.
 - Desventajas: ineficiente para buscar bloques contiguos; acceso lento (cada búsqueda requiere leer bloques libres del disco); si se corrompe un puntero, se pierde parte de la lista.
- Agrupamiento:**
- Variante de la lista enlazada.

- En lugar de que cada bloque libre apunte a uno solo, un bloque libre contiene la dirección de los siguientes N bloques libres y un puntero a otro bloque que contiene el siguiente grupo.
- Se reducen los accesos al disco.
- Ventajas: búsqueda mucho más rápida que en la lista simple; menos accesos al disco para localizar bloques libres; sigue siendo simple de implementar.
- Desventajas: todavía es difícil encontrar grandes áreas contiguas; depende de la integridad de los bloques de grupo.

Recuento:

- Se observa que los bloques libres suelen aparecer en grupos contiguos.
- Se guarda una lista de entradas de la forma: (bloque inicial, cantidad de bloques contiguos libres).
- Permite representar grandes rangos de bloques libres con un solo registro.
- Ventajas: excelente para sistemas en los que los bloques libres están agrupados; permite encontrar áreas contiguas rápidamente; representa muchos bloques libres con pocos registros; muy eficiente en sistemas grandes.
- Desventajas: más complejo de mantener cuando los archivos se crean y borran continuamente; menos eficiente si los bloques libres están muy fragmentados y no contiguos.

Técnicas para la administración de la memoria de un archivo:

Asignación contigua:

- Cada archivo ocupa bloques consecutivos en el disco.
- Se almacena el bloque inicial y el tamaño.
- La lectura es secuencial y extremadamente rápida.
- Ventajas: excelente performance en acceso secuencial y aleatorio; fácil de administrar.
- Desventajas: necesita conocer el tamaño del archivo por adelantado; difícil crecimiento del archivo.
- Puede producir fragmentación interna (último bloque puede quedar parcialmente lleno) y fragmentación externa (requiere bloques contiguos).

Asignación enlazada:

- El archivo se guarda como una lista enlazada de bloques en cualquier parte del disco.
- Cada bloque contiene un puntero al siguiente bloque.
- No requiere espacio contiguo.
- Ventajas: Los archivos pueden crecer dinámicamente; no requiere bloques contiguos.
- Desventajas: acceso aleatorio muy lento (hay que recorrer la lista); overhead por punteros; si se corrompe un puntero, se pierde parte del archivo.

• Puede producir fragmentación interna (cada bloque puede no usarse por completo). **Asignación indexada:**

- Se crea un bloque índice (index block) para cada archivo.
- El bloque índice contiene una tabla de punteros a los bloques de datos.
- Los bloques pueden estar dispersos como en la asignación enlazada, pero se accede directo mediante el índice.
- Ventajas: permite acceso directo y secuencial; archivos pueden crecer sin necesidad de contigüidad; no depende de la integridad de una cadena de punteros, como en la asignación enlazada.
- Desventajas: ocupa espacio extra para el bloque índice; el tamaño del archivo puede estar limitado por la cantidad de punteros del índice; ligeramente, más compleja de administrar.
- Puede producir fragmentación interna (tamaño del último bloque + tamaño del bloque índice).

Organización del Filesystems UNIX SYSTEM V

Superblock: contiene información global del FS (tamaño, número de i-nodos, contadores, punteros a listas libres/bitmaps, estado, etc.).

Árbol de directorios: el FS tiene un único *root* /. Los directorios son archivos especiales que contienen entradas con nombre $\Rightarrow$ num_i_node.

Tabla de i-nodos: espacio (fijo) donde están almacenados todos los i-nodos; cada i-node tiene metadatos y punteros a bloques (los 13 punteros). En System V el número de i-nodos suele fijarse al crear el FS.

Bloques de datos (data blocks / clusters): contienen los bytes reales de los archivos. La unidad de asignación puede ser un bloque o un cluster (varios sectores).

Estructuras de gestión de espacio libre: bitmaps o listas libres para bloques e i-nodos; System V clásico usa estructuras y contadores en el superblock y a nivel de disco.

Directorio = mapeo nombre $\rightarrow$ i-node: operaciones como crear/renombrar/borrar actúan modificando entradas de directorio (no el i-node del archivo salvo cuando cambian metadatos).

Enlaces: un enlace duro incrementa el contador de enlaces del i-node; un enlace simbólico es un archivo especial con contenido que es la ruta.

Asignación en disco: System V usa el esquema de i-node con punteros directos/indirectos (descritos arriba), por eso el límite de tamaño está dado por los niveles de indirectos.

Caching y Buffer Cache: para eficiencia, se usan buffers en memoria (buffer caché) y caché de páginas/inodes para evitar accesos físicos frecuentes.

Memoria virtual es una técnica de administración de memoria que permite al SO simular que tiene más memoria RAM de la que hay físicamente instalada. Funciona creando un espacio de direcciones lógicas que abstraen el HW. Se apoya en 3 conceptos:

SWAPP(espacio de intercambio): Está ubicada como una partición en la memoria secundaria, usada para mover páginas inactivas desde la RAM

Paginación: La memoria RAM se divide en bloques de tamaño fijo llamado páginas

Beneficios Principales:

- **Aislamiento:** Cada proceso cree que tiene todo el rango de memoria para sí mismo, evitando que un programa sobrescriba los datos de otro.
- **Multitarea:** Permite ejecutar más aplicaciones simultáneamente de las que cabrían en la RAM física.
- **Eficiencia:** El sistema solo carga en RAM las partes del programa que realmente se están ejecutando en ese momento.

Memoria virtual paginada son todas las páginas que pertenecen al espacio de direcciones de un proceso, estén en RAM o estén en swap.

Asignación fija en esta política el kernel le da una cantidad fija de frames a un proceso desde que comienza hasta que finaliza. Si se le asigna pocos frames produce fallos de página

Con reparto equitativo: Si hay 100 marcos disponibles en memoria y hay 5 procesos entonces el kernel le asigna 20 frames a cada proceso sin importar su tamaño, su comportamiento.

- Genera fragmentación interna para procesos pequeños
- Genera fallos de páginas para procesos grandes
- Puede generar hiperpaginación

Con reparto proporcional: Cada proceso obtiene su marcos acorde al tamaño del proceso. Si hay 100 frames y el proceso A ocupa 20% y el proceso B ocupa 80% el kernel le dará más frame al proceso B.

- Genera fragmentación interna para procesos pequeños
- Puede generar hiperpaginación

Asignación dinámica en esta política el kernel le da mas frames a procesos que producen fallos de página y le quita frame a procesos que no utilizan muchos frames

Reemplazo local ocurre cuando el proceso tiene todos sus frames ocupados y hay un solo fallo de página entonces el kernel busca la página víctima dentro del mismo proceso. Puede producir una hiperpaginación si no se controla.

Reemplazo global ocurre cuando hay un fallo de página en un proceso entonces el kernel selecciona cualquier frame libre de la RAM y si no encuentra usa el algoritmo de reemplazo para buscar la página víctima. Puede

producir una hiperpaginación si no se controla, más probable que reemplazo local.

Reemplazo global $\Rightarrow$ asignación dinámica, el kernel busca un frame en toda la RAM

Reemplazo local $\Rightarrow$ asignación fija, el kernel busca un frame libre en los Frames del proceso

Hiperpaginación (trashing) se produce cuando el SO pasa todo el tiempo atendiendo fallos de página y no ejecutando instrucciones útiles.

Causantes:

- 1) cada proceso tiene menos marcos asignados que su working set real
- 2) Aumentar demasiado el grado de multiprogramación
- 3) Mala política de reemplazo

El SO detecta trashing por 2 mecanismos:

PFF – Page Fault Frequency (Frecuencia de fallos) si la frecuencia es alta entonces hay trashing. Si la frecuencia es baja puede devolverse frames. El kernel soluciona aumentando marcos asignados al proceso sino suspenden el proceso para liberar marcos (swapping out)

Working Set (Conjunto de trabajo) es la cantidad mínima de frames que un proceso necesita tener para no producir fallos de páginas en un intervalo de tiempo. Si el tamaño del working set es mayor a la cantidad de marcos asignados. El kernel soluciona reduciendo el grado de multiprogramación swapeando el proceso, esto quiere decir que el kernel busca un proceso con baja prioridad y lo quita de la RAM hacia el área SWAPP para liberar recursos, frames.

PFF VS WORKING SET:

- **PFF** Controla la frecuencia de fallos de página de cada proceso. Si un proceso tiene muchos fallos, necesita más marcos. Si un proceso tiene muy pocos fallos, se le pueden quitar marcos. Cada proceso debe mantener su page fault rate dentro de un rango aceptable.
- Esta técnica permite determinar la existencia de thrashing de la siguiente manera: o Si muchos procesos empiezan a tener frecuencias de fallos muy altas al mismo tiempo, significa que todos están compitiendo por muy pocos marcos, el disco está siendo usado constantemente y el CPU queda ocioso. o Entonces, el sistema operativo concluye que hay thrashing.
- En conclusión, esta técnica detecta thrashing cuando la frecuencia de fallos crece por encima de un umbral. Es una técnica reactiva, basada en el comportamiento del page fault rate.
- **WS:** Cada proceso tiene un conjunto de páginas activas en un intervalo reciente de tiempo. Ese conjunto es su Working Set. El sistema operativo debe asignar, al menos, tantos marcos como páginas tenga su Working Set.
- Esta técnica permite determinar la existencia de thrashing de la siguiente manera: o Se suman los tamaños de los Working Sets de todos los procesos. o Si la suma es mayor que la memoria física, el sistema no puede satisfacer la demanda real de memoria. o Entonces, inevitablemente, ocurre thrashing.
- En conclusión, esta técnica detecta thrashing cuando la memoria no alcanza para sostener los Working Sets activos. Es una técnica basada en la localidad y en el uso real.

2. El efecto del Δ (Delta)

Tamaño de Δ : Muy Pequeño

- **Efecto en la Localidad:** No llega a "cubrir" toda la localidad. El sistema olvida páginas que el proceso todavía está usando.
- **Consecuencia:** Alta tasa de fallos de página. El proceso pide una página que el SO acaba de quitar por error.

Tamaño de Δ : Muy Grande

- **Efecto en la Localidad:** Cubre la localidad actual y también la anterior (páginas viejas).
- **Consecuencia:** Desperdicio de RAM. Mantiene páginas que el proceso ya no usa (que no forman parte de su localidad actual).

Tamaño de Δ : Ideal

- **Efecto en la Localidad:** Coincide con el tamaño de la localidad actual.
- **Consecuencia:** Rendimiento óptimo. El proceso tiene lo que necesita y nada más.

Ejemplo de funcionamiento

1. El Kernel calcula: "El Working Set de este proceso hoy es de 10 páginas".
 2. El Kernel **le asigna 10 frames** (si hay disponibles).
- **Nota explicativa:** Si el Δ es muy chico, el Kernel calculará (erróneamente) que el proceso solo necesita 2 páginas. Le asignará solo 2 frames, y el proceso empezará a fallar porque su localidad real era de 10.

Algoritmos de selección de página víctima:

(peor) FIFO —Second-Chance —LRU(mejor en practica) —OPT(teoricamente) (mejor)

OPT(óptimo) selecciona una página víctima cuya próxima referencia se encuentre más lejos de la actual.

Ventajas con OPT: Minimiza la cantidad total de fallos de página. No se realiza en tiempo real.

LRU selecciona la página víctima menos reciente usada. Hace uso del bit R(referencia), si R=0 entonces se considera un acceso viejo a la página. También analiza el instante de su última referencia o una pila.

Ventajas: aprovecha la localidad temporal(se usó recién, se vuelve a usar), páginas usadas recientemente pueden usarse de pronto. Minimiza la cantidad total de fallos no tanto como OPT.

Desventaja: Costosa en HW(requiere acceso a la MMU).

Second Chance Selecciona como página víctima la página más vieja en memoria.utiliza un bit de referencia R, mantiene los marcos en una lista circular. Tiene acceso FIFO

Si R=0 entonces selecciona esa página como víctima

Si R=1 significa que esa página se accedió recientemente y le da una segunda oportunidad, cambia el bit R=0 para elegirlo si vuelve a seleccionar. Costo favorable.

Marco/Página	1	2	15	4	6	2	1	5	6	10	4	6	7	9	1
F1	1	1	1	1	1	1	1*	1	1	1	4	4	4	4	4
F2		2	2	2	2	2*	2*	2	2	2	2	2	7	7	7
F3			15	15	15	15	15	5	5	5	5	5	5	9	9
F4				4	4	4	4	4	4	10	10	10	10	10	1
F5					6	6	6	6	6*	6*	6	6*	6*	6*	6*
¿PF?	X	X	X	X	X			X		X	X		X	X	X

QUEUE: 1* 2* 15 4 6 1 2 5 10 6* 4 7 9 1

Las páginas que llegan van ocupando los frames disponibles hasta ocupar todos y luego se producen los fallos de páginas si llegan páginas nuevas. Cuando llega una pagina y se encuentra en algún frame entonces se pone el * representando el bit R=1.

Al momento de elegir una pagina víctima se recorre la queue hasta encontrar un número que no tenga * y en su recorrido se quitan los * de los que sí tenían y se colocan al final de la queue y después el número de la página nueva, en ese orden. Se conservan el resto de números que tenían * y que están fuera de su recorrido.

FIFO selecciona la página que llegó primero en la cola y así sucesivamente

Marco/Página	1	2	15	4	6	2	1	5	6	10	4	6	7	9	1
F1	1	1	1	1	1	1	1	5	5	5	5	5	5	5	5
F2		2	2	2	2	2	2	2	2	10	10	10	10	10	10
F3			15	15	15	15	15	15	15	15	15	15	7	7	7
F4				4	4	4	4	4	4	4	4	4	4	9	9
F5					6	6	6	6	6	6	6	6	6	6	1
¿PF?	X	X	X	X	X			X		X			X	X	X

Queue: 1, 2, 15, 4, 6, 5, 10, 7, 9, 1,

Las páginas que llegan van ocupando los frames disponibles hasta ocupar todos y luego se producen los fallos de páginas si llegan páginas nuevas. Para el reemplazo toma el primer elemento de la queue y lo elimina de la fila

Algoritmos de planificación de discos:

El objetivo de estas planificaciones es minimizar el movimiento del cabezal, ordenando lógicamente los requerimientos que están en la queue.

La atención de requerimientos a pistas duplicadas se resuelven según el algoritmo de planificación.

Ejemplo: Tengo lotes de requerimientos: {10,2,3,10}, para FCFS lo atiendo tantas veces como requiera(en este caso 2 veces), para los demás algoritmos se atienden de manera consecutiva (una sola vez).

FCFS Atiende los requerimientos en el mismo orden en que llegan

Sirve para sistemas donde la prioridad es la simplicidad.

Sirve para evitar inanición (todos se atienden seguro).

SSTF Siempre atiende el requerimiento más cercano a la posición actual del cabezal. Minimiza el tiempo de búsqueda (seek). Genera inanición, los requerimientos pueden no ser atendidos por mucho tiempo.

Sirve cuando se busca optimizar el tiempo de acceso al disco.

SCAN recorre las pistas en una dirección atendiendo pedidos en su camino. Al llegar al extremo(última pista), invierte el sentido y atiende los pedidos del otro lado.

Sirve para Sistemas donde se quiere buena performance y evitar estar siempre en un lado del disco.

LOOK NO va hasta el final del disco. Solo va hasta la última solicitud en esa dirección, luego vuelve. Evita movimientos innecesarios al final del disco. Más eficiente que SCAN.

C-SCAN (alg. circular) Atiende solo en una dirección. Cuando llega a la pista final, vuelve a la pista inicial del disco para atender los requerimientos restantes. Evita inanición mejor que SSTF.

Sirve en discos reales por su uniformidad.

C-LOOK (alg. circular) Atiende solo en una dirección. Cuando llega a la última solicitud(pista final), vuelve a la pista inicial (requerimiento más grande) para atender los requerimientos restantes.

- Una vez que no existan más requerimientos por *page faults* en la cola, se procede:
 - **FCFS**: en orden *FCFS*
 - **SSTF**: en orden *SSTF*
 - **SCAN**: con el sentido que determina la atención de los últimos dos requerimientos → puede cambiar de sentido
 - **C-SCAN**: con el sentido original → el sentido no cambia
 - **LOOK**: del mismo modo en que lo hace el *SCAN*
 - **C-LOOK**: del mismo modo en que lo hace el *C-SCAN*

FCFS termina de atender PF y sigue con los req de la cola

SSTF termina de atender PF y sigue buscando req cercanos al último PF

SCAN termina de atender PF y sigue con el sentido del último PF

LOOK lo mismo que SCAN

C-SCAN y C-LOOK terminan de atender PF y sigue con el sentido del principio, no cambia nunca su sentido.

Sectores:

Identificación antigua (C,H,S) cilindro, heads(cabezas), sectores

C : (0,cant cilindro-1)

H: (0,cant cabezas-1)

S: (1, cant sectores por pista)

Identificación moderna LBA: (0, cant sectores-1)

Traducción de (C,H,S) a LBA

$$A = (C * N_{heads} + H) * N_{sectores} + (S - 1)$$

A es el valor LBA a obtener

→ N_{heads} es la cantidad de cabezas lecto/grabadoras

→ $N_{sectores}$ es la cantidad de sectores por pista

Traducción de (C,H,S) a partir del LBA

$C = LBA \div (N_{heads} * N_{sectores})$	$H = \text{Resto de } (LBA \bmod (N_{heads} * N_{sectores})) / N_{sectores}$	$S = (LBA \bmod (N_{heads} * N_{sectores})) \bmod N_{sectores} + 1$
---	--	---

Seek time(posicionamiento) es el tiempo que tarda el cabezal del disco en moverse hasta la pista requerida donde se encuentran los datos. Es el componente más lento y depende de la distancia que debe recorrer el cabezal.

Latency time(latencia) es el tiempo que tarda el disco en girar sobre su eje hasta que el sector deseado quede debajo del cabezal. Depende de la velocidad de rotación del disco (RPM).

A mayor velocidad menor latencia.

Transfer time es el tiempo necesario para leer o escribir los datos una vez que el cabezal ya está en el lugar correcto. Es el más rápido de los tres y depende del ancho de banda del disco.

Las metas que debe perseguir un sistema operativo para la administración de la entrada/salida son:

- **Eficiencia:** Aprovechar al máximo los dispositivos, reduciendo tiempos de espera y evitando que el CPU quede bloqueado.

- **Equidad:** Asegurar que todos los procesos tengan acceso razonable a los dispositivos, evitando que uno monopolice la E/S.
- **Independencia del dispositivo:** Que los programas puedan usar dispositivos sin conocer sus detalles físicos (drivers, diferencias de hardware, etc.).
- Uniformidad en la interfaz: Proveer una forma común de acceder a distintos dispositivos, simplificando el trabajo del programador.
- **Concurrencia y multiplexación:** Permitir que varios procesos usen dispositivos simultáneamente o que el sistema operativo intercale peticiones de forma ordenada.
- **Manejo de errores:** Detectar, informar y, si es posible, recuperar errores de dispositivos o transferencias de E/S.
- **Protección y seguridad:** Evitar accesos indebidos a dispositivos o datos y garantizar que un proceso no interfiera con otro.
- **Buffering y manejo eficiente de memoria:** Utilizar buffers para optimizar la velocidad entre CPU, memoria y dispositivos más lentos.

Drivers

Son programas de bajo nivel que están dentro del kernel y permiten que el SO se comunice con un dispositivo de hardware. Actúan como traductores entre el sistema operativo y el dispositivo, ocultando los detalles específicos del hardware.

¿Qué funciones mínimas deben proveer?

- **Inicialización del dispositivo:** Configurarlos para que quede listo para operar.
- **Lectura y escritura de datos:** Proveer rutinas para enviar datos al dispositivo o recibirlos de él.
- **Manejo de interrupciones:** Responder cuando el dispositivo genera una interrupción.
- Control del dispositivo: Operaciones especiales (cambios de modo, comandos específicos, reset, etc.).
- **Reporte y manejo de errores:** Detectar fallas y comunicarlas al sistema operativo.

¿Quién determina cuáles deben ser estas funciones?

El que determina cuáles deben ser estas funciones es el sistema operativo, que establece la API o interfaz estándar que todos los drivers deben implementar.

Pasos de un proceso al realizar un requerimiento:

Requerimiento: lectura de un archivo ubicado en disco (Eje 1)

- El proceso llama a Read() , se ejecuta una syscall, pasa al modo kernel
- El kernel identifica qué dispositivo usar y ejecuta su driver correspondiente
- A su vez el kernel consulta en la tabla de archivos y buffer para ver si la información se encuentra ahí o debe ir al dispositivo.
- En caso de no encontrar la información en el buffer caché o la tabla de archivos. El kernel arma la solicitud de operación (dirección de disco, lectura o escritura, cantidad de bytes, LBA, etc)
- El kernel invoca al driver y el driver traduce la solicitud al formato del HW.
- El driver programa al HW ejecutar la operación por DMA o E/S programada o interrupción(según corresponda)
- El proceso cambia su estado a "waiting" y queda en la cola de procesos bloqueados por E/S.
- El hw realiza la operación mueve, cabezas, busca cilindros, etc.
- Finaliza la operación y el HW manda una interrupción de E/S a la CPU
- El kernel atiende la interrupción y copia los datos al buffer
- El kernel cambia el estado del proceso a "listo" y pasa al modo usuario ese proceso y read() devuelve los datos

Ejemplo 2:

1. El proceso solicita la operación: El proceso ejecuta una llamada al sistema (por ejemplo, *read()*). Esto pasa del modo usuario al modo *Kernel*.
2. El Subsistema de E/S recibe el pedido: El *Kernel* verifica qué dispositivo es, si está disponible, si necesita usar un *buffer* y si la operación requiere acceso bloqueante o no. Crea una solicitud de E/S y la coloca en la cola del dispositivo.
3. Se invoca al *driver* del dispositivo: El subsistema de E/S llama al *driver* correspondiente (por ejemplo, *driver* de disco). El *driver*: traduce el pedido del sistema operativo en comandos específicos del dispositivo; prepara los parámetros que necesita el *hardware* (sector, dirección, tamaño, etc.).
4. El *driver* ordena la operación al controlador del dispositivo: El *driver* escribe, en los registros del controlador (*hardware*), los datos necesarios (dirección del bloque a leer, cantidad de bytes, dirección de memoria donde almacenar el resultado).
5. El controlador ejecuta la operación: El controlador de disco toma los comandos y los envía al dispositivo físico. El dispositivo inicia la operación (por ejemplo, mover cabezales y leer el sector).
6. El dispositivo realiza la operación: El disco lee el bloque solicitado y, cuando termina, envía una interrupción al CPU avisando que la lectura finalizó.
7. El *driver* atiende la interrupción: El CPU detiene lo que estaba haciendo y ejecuta la rutina de interrupción del *driver*. El *driver*: confirma que la operación terminó; copia (o valida) los datos en el *buffer* del sistema operativo; marca la solicitud como completada.
8. El Subsistema de E/S devuelve el resultado al proceso: El *Kernel* despierta o reanuda al proceso que estaba esperando el dato. El proceso recibe el bloque leído y continúa ejecutando.

Servicios que ofrece el kernel para la administración de E/S

Buffering: almacenamiento de los datos en memoria mientras se transfieren.

Solucionar problemas de velocidad entre los dispositivos.

Solucionar problemas de tamaño y/o forma de los datos entre los dispositivos.

Caching: mantener en memoria copia de los datos de reciente acceso para mejorar la performance.

Spooling: administrar la cola de requerimientos de un dispositivo.

Maneja dispositivos compartidos (como impresoras), almacenando trabajos en cola

Técnicas de E/S programa trabajan de 2 formas:

E/S mapeada: En esta técnica, los registros de los dispositivos de E/S (por ejemplo, “estado”, “control”, “dato”) se asignan a direcciones dentro del espacio de memoria del sistema. Esto significa que:

- El CPU lee y escribe en los dispositivos usando las mismas instrucciones que usa para acceder a la memoria (LOAD, STORE).
- No existe una instrucción especial para E/S.
- El hardware del sistema distingue si una dirección corresponde a memoria real o a un dispositivo.

E/S aislada: En este caso, los dispositivos no comparten el espacio de direcciones de la memoria. Tienen un espacio separado, llamado “puertos de E/S”. Para acceder a esos puertos:

- El CPU debe usar instrucciones especiales (por ejemplo, IN y OUT en la arquitectura x86).
- La E/S está “aislada” y separada del acceso a memoria común.

Técnicas de E/S:

E/S programada: El CPU verifica, continuamente (polling), si el dispositivo está listo y copia los datos(byte a byte) entre el dispositivo y la memoria. El CPU trabaja todo el tiempo.

E/S dirigida por interrupciones: El CPU le pide al dispositivo que haga la operación y sigue ejecutando otras tareas. Cuando el dispositivo termina, envía una interrupción y el CPU copia los datos(byte a byte) desde el registro de datos del módulo E/S hacia la memoria RAM. El CPU trabaja sólo al principio y al final.

DMA (Acceso Directo a Memoria): Un módulo especial (el controlador DMA) transfiere datos entre la memoria y el dispositivo y viceversa sin intervención del CPU. El CPU sólo inicia la operación y el DMA hace todo.

Dispositivos Orientados a bloques:

- Transfieren datos en bloques de tamaño fijo (por ejemplo, 512 bytes, 4 KB).
- Acceso aleatorio: Se puede leer/escribir, directamente, en cualquier bloque sin recorrer los anteriores.
- Son ideales para almacenamiento donde es importante acceder rápidamente, a posiciones específicas.
- Típicamente, gestionados por el sistema operativo como un conjunto de bloques numerados.

Dispositivos Orientados a flujos:

- Transfieren datos como un flujo continuo de bytes, sin estructura fija.
- Acceso secuencial: Para llegar a cierto punto del flujo, hay que pasar por los datos anteriores.
- El sistema no maneja bloques, sino un flujo tipo “tubería” de datos.
- Se usan para datos que se consumen o producen de manera continua.

Buffer cache

En system V, los buffers están en una lista libre(free list) o una hash queue, un buffer puede cambiar de hash cuando cambia el bloque que representa